Programmation Par Contraintes et Applications

Yahia Lebbah

Université Oran1, Algeria
Email: ylebbah@gmail.com
Web: https://github.com/ylebbah/teaching/constraints

October 24, 2026

1 Satisfaction de contraintes
- La notation CSP et définitions de base
- Algorithmes de Filtrage
  - Algorithmes de Filtrage - *arc*-consistance
  - Algorithmes de Filtrage - Contraintes Globales
  - Algorithmes de Filtrage - Consistances fortes
- Résolution des CSPs
  - Résolution - Backtrack intelligent
  - Résolution - Le problème SAT

2 Optimisation
- Rigueur et analyse par intervalles
- Algorithme Branch& Bound
- Evaluation inférieure
- Evaluation supèrieure
- Réduction des domaines
- Optimisation locale

3 Environnements de programmation

Origine

La programmation par contraintes (PPC) est issue d'un rapprochement entre :

- Satisfaction de contraintes : Le système Alice - [*J.L. Laurière, A Language and a Program for Stating and Solving Combinatorial Problems, AIJ 1978*]

Origine

La programmation par contraintes (PPC) est issue d'un rapprochement entre :

- Satisfaction de contraintes : Le système Alice - [*J.L. Laurière, A Language and a Program for Stating and Solving Combinatorial Problems, AIJ 1978*]
  - Absence de résultats pratiques de la PNE !

Origine

La programmation par contraintes (PPC) est issue d'un rapprochement entre :

- Satisfaction de contraintes : Le système Alice - [*J.L. Laurière, A Language and a Program for Stating and Solving Combinatorial Problems, AIJ 1978*]
  - Absence de résultats pratiques de la PNE !
  - Idée : Recours aux démarches heuristiques et aux raisonnements locaux, tout en guarantissant la globalité !

Origine

La programmation par contraintes (PPC) est issue d'un rapprochement entre :

- Satisfaction de contraintes : Le système Alice - [*J.L. Laurière, A Language and a Program for Stating and Solving Combinatorial Problems, AIJ 1978*]
  - Absence de résultats pratiques de la PNE !
  - Idée : Recours aux démarches heuristiques et aux raisonnements locaux, tout en guarantissant la globalité !
- Recherche opérationnelle : PL (198?, ...), Graphes (1994, ...), PLNE (199?, ...), PNLNE (200?, ...), ...

Origine

La programmation par contraintes (PPC) est issue d'un rapprochement entre :

- Satisfaction de contraintes : Le système Alice - [*J.L. Laurière, A Language and a Program for Stating and Solving Combinatorial Problems, AIJ 1978*]
  - Absence de résultats pratiques de la PNE !
  - Idée : Recours aux démarches heuristiques et aux raisonnements locaux, tout en guarantissant la globalité !
- Recherche opérationnelle : PL (198?, ...), Graphes (1994, ...), PLNE (199?, ...), PNLNE (200?, ...), ...
  - 1ère Idée : RO pour améliorer la PPC

Origine

La programmation par contraintes (PPC) est issue d'un rapprochement entre :

- Satisfaction de contraintes : Le système Alice - [*J.L. Laurière, A Language and a Program for Stating and Solving Combinatorial Problems, AIJ 1978*]
  - Absence de résultats pratiques de la PNE !
  - Idée : Recours aux démarches heuristiques et aux raisonnements locaux, tout en guarantissant la globalité !
- Recherche opérationnelle : PL (198?, ...), Graphes (1994, ...), PLNE (199?, ...), PNLNE (200?, ...), ...
  - 1ère Idée : RO pour améliorer la PPC
  - 2ème Idée : PPC pour améliorer la RO

$$\text{PPC} = \text{Contraintes} + \text{Résolution}$$

La notation CSP

Un CSP (Constraint Satisfaction Problem) $\mathcal{P}$ est un triplet $\langle \mathcal{X}, \mathcal{D}, \mathcal{C} \rangle$, avec :

- $\mathcal{X}$ un ensemble de n variables $x_1, \ldots, x_n$.

La notation CSP

Un CSP (Constraint Satisfaction Problem) $\mathcal{P}$ est un triplet $\langle \mathcal{X}, \mathcal{D}, \mathcal{C} \rangle$, avec :

- $\mathcal{X}$ un ensemble de n variables $x_1, \ldots, x_n$.
- $\mathcal{D}$ le n-uplet $\langle D_1, \ldots, D_n \rangle$ des domaines des variables. D_i est l'ensemble contenant les valeurs de la variable x_i.

$$x_i \in D_i, i = 1..n.$$

La notation CSP

Un CSP (Constraint Satisfaction Problem) $\mathcal{P}$ est un triplet $\langle \mathcal{X}, \mathcal{D}, \mathcal{C} \rangle$, avec :

- $\mathcal{X}$ un ensemble de n variables $x_1, \ldots, x_n$.
- $\mathcal{D}$ le n-uplet $\langle D_1, \ldots, D_n \rangle$ des domaines des variables. D_i est l'ensemble contenant les valeurs de la variable x_i.

$$x_i \in D_i, i = 1..n.$$

- $\mathcal{C} = \{C_1, \ldots, C_m\}$ l'ensemble des contraintes.
  Sémantiquement

$$C_i(x_{i_1}, \ldots, x_{i_l}) \subseteq D_{i_1} \times \ldots \times D_{i_l}.$$

Exemple : cryptarithmétique

Remplacer chaque lettre par une décimale telle que la somme

$$\begin{array}{rr} & S\ E\ N\ D \\ + & \\ & M\ O\ R\ E \\ \hline = & M\ O\ N\ E\ Y \end{array}$$

soit correcte.

Exemple : cryptarithmétique

Remplacer chaque lettre par une décimale telle que la somme

$$\begin{array}{r} \,S\;E\;N\;D \\ +\;\;M\;O\;R\;E \\ \hline =\;M\;O\;N\;E\;Y \end{array}$$

soit correcte.
Il existe une solution unique $9567 + 1085 = 10652$.

Exemple : cryptarithmétique - CSP

$$\mathcal{X} = \{S, E, N, D, M, O, R, Y, R_1, R_2, R_3, R_4\}$$

Exemple : cryptarithmétique - CSP

$$\begin{aligned}
\mathcal{X} = & \{S, E, N, D, M, O, R, Y, R_1, R_2, R_3, R_4\} \\
\mathcal{D} = & \{D_S, D_E, D_N, D_D, D_M, D_O, D_R, D_Y, D_{R_1}, D_{R_2}, D_{R_3}\} \\
& D_S = D_E = D_N = D_D = D_M = D_O = D_R = D_Y = [0...9], \\
& D_{R_1} = D_{R_2} = D_{R_3} = [0...1]
\end{aligned}$$

Exemple : cryptarithmétique - CSP

$$
\begin{aligned}
\mathcal{X} = & \quad \{S, E, N, D, M, O, R, Y, R_1, R_2, R_3, R_4\} \\
\mathcal{D} = & \quad \{D_S, D_E, D_N, D_D, D_M, D_O, D_R, D_Y, D_{R_1}, D_{R_2}, D_{R_3}\} \\
& \quad D_S = D_E = D_N = D_D = D_M = D_O = D_R = D_Y = [0...9], \\
& \quad D_{R_1} = D_{R_2} = D_{R_3} = [0...1] \\
\mathcal{C} = & \quad \{C_1, ..., C_8\} \\
C_1 : & \quad S \neq E \neq N \neq D \neq M \neq O \neq R \neq Y
\end{aligned}
$$

Exemple : cryptarithmétique - CSP

$$
\begin{aligned}
\mathcal{X} = & \quad \{S, E, N, D, M, O, R, Y, R_1, R_2, R_3, R_4\} \\
\mathcal{D} = & \quad \{D_S, D_E, D_N, D_D, D_M, D_O, D_R, D_Y, D_{R_1}, D_{R_2}, D_{R_3}\} \\
& \quad D_S = D_E = D_N = D_D = D_M = D_O = D_R = D_Y = [0...9], \\
& \quad D_{R_1} = D_{R_2} = D_{R_3} = [0...1] \\
\mathcal{C} = & \quad \{C_1, ..., C_8\} \\
C_1 : & \quad S \neq E \neq N \neq D \neq M \neq O \neq R \neq Y \\
C_2 : & \quad S \neq 0; \quad C_3 : M \neq 0 \quad C_4 : R_1 = M
\end{aligned}
$$

Exemple : cryptarithmétique - CSP

$$
\begin{aligned}
\mathcal{X} = & \quad \{S, E, N, D, M, O, R, Y, R_1, R_2, R_3, R_4\} \\
\mathcal{D} = & \quad \{D_S, D_E, D_N, D_D, D_M, D_O, D_R, D_Y, D_{R_1}, D_{R_2}, D_{R_3}\} \\
& \quad D_S = D_E = D_N = D_D = D_M = D_O = D_R = D_Y = [0...9], \\
& \quad D_{R_1} = D_{R_2} = D_{R_3} = [0...1] \\
\mathcal{C} = & \quad \{C_1, ..., C_8\} \\
C_1 : & \quad S \neq E \neq N \neq D \neq M \neq O \neq R \neq Y \\
C_2 : & \quad S \neq 0; \quad C_3 : M \neq 0 \quad C_4 : R_1 = M \\
C_5 : & \quad R_2 + S + M = O + 10 \times R_1
\end{aligned}
$$

Exemple : cryptarithmétique - CSP

$$
\begin{aligned}
\mathcal{X} = & \quad \{S, E, N, D, M, O, R, Y, R_1, R_2, R_3, R_4\} \\
\mathcal{D} = & \quad \{D_S, D_E, D_N, D_D, D_M, D_O, D_R, D_Y, D_{R_1}, D_{R_2}, D_{R_3}\} \\
& \quad D_S = D_E = D_N = D_D = D_M = D_O = D_R = D_Y = [0...9], \\
& \quad D_{R_1} = D_{R_2} = D_{R_3} = [0...1] \\
\mathcal{C} = & \quad \{C_1, ..., C_8\} \\
C_1 : & \quad S \neq E \neq N \neq D \neq M \neq O \neq R \neq Y \\
C_2 : & \quad S \neq 0; \quad C_3 : M \neq 0 \quad C_4 : R_1 = M \\
C_5 : & \quad R_2 + S + M = O + 10 \times R_1 \\
C_6 : & \quad R_3 + E + O = N + 10 \times R_2
\end{aligned}
$$

Exemple : cryptarithmétique - CSP

$$
\begin{aligned}
\mathcal{X} = & \quad \{S, E, N, D, M, O, R, Y, R_1, R_2, R_3, R_4\} \\
\mathcal{D} = & \quad \{D_S, D_E, D_N, D_D, D_M, D_O, D_R, D_Y, D_{R_1}, D_{R_2}, D_{R_3}\} \\
& \quad D_S = D_E = D_N = D_D = D_M = D_O = D_R = D_Y = [0...9], \\
& \quad D_{R_1} = D_{R_2} = D_{R_3} = [0...1] \\
\mathcal{C} = & \quad \{C_1, ..., C_8\} \\
C_1 : & \quad S \neq E \neq N \neq D \neq M \neq O \neq R \neq Y \\
C_2 : & \quad S \neq 0; \quad C_3 : M \neq 0 \quad C_4 : R_1 = M \\
C_5 : & \quad R_2 + S + M = O + 10 \times R_1 \\
C_6 : & \quad R_3 + E + O = N + 10 \times R_2 \\
C_7 : & \quad R_4 + N + R = E + 10 \times R_3
\end{aligned}
$$

Exemple : cryptarithmétique - CSP

$$
\begin{aligned}
\mathcal{X} = & \quad \{S, E, N, D, M, O, R, Y, R_1, R_2, R_3, R_4\} \\
\mathcal{D} = & \quad \{D_S, D_E, D_N, D_D, D_M, D_O, D_R, D_Y, D_{R_1}, D_{R_2}, D_{R_3}\} \\
& \quad D_S = D_E = D_N = D_D = D_M = D_O = D_R = D_Y = [0...9], \\
& \quad D_{R_1} = D_{R_2} = D_{R_3} = [0...1] \\
\mathcal{C} = & \quad \{C_1, ..., C_8\} \\
C_1 : & \quad S \neq E \neq N \neq D \neq M \neq O \neq R \neq Y \\
C_2 : & \quad S \neq 0; \quad C_3 : M \neq 0 \quad C_4 : R_1 = M \\
C_5 : & \quad R_2 + S + M = O + 10 \times R_1 \\
C_6 : & \quad R_3 + E + O = N + 10 \times R_2 \\
C_7 : & \quad R_4 + N + R = E + 10 \times R_3 \\
C_8 : & \quad D + E = Y + 10 \times R_4
\end{aligned}
$$

Exemple : emploi du temps

Définir l'emploi du temps de managers d'un centre d'assistance comportant 5 activités, représentés par un ensemble $A = \{$**M***atin*, **J***our*, **A***près-midi*, **S***oir*, **W***eek-end*$\}$, 7 personnes, représentés par l'ensemble $P = \{$*Hassen, Yassine, Ahmed, Tarik, Omar, Saïd, Ali*$\}$. Une personne de P doit effectuer au moins une des activiés de A. On veut affecter les activités aux managers en respectant le fait que :

- Au moins une personne doit faire M, et au plus deux.
- Au moins une personne doit faire J, et au plus deux.
- Au moins une personne doit faire A, et pas plus d'une.
- Au plus deux personnes peuvent faire S.
- Au plus deux personnes peuvent faire W.

Exemple : emploi du temps

Définir l'emploi du temps de managers d'un centre d'assistance comportant 5 activités, représentés par un ensemble $A = \{$**M***atin,* **J***our,* **A***près-midi,* **S***oir,* **W***eek-end*$\}$, 7 personnes, représentés par l'ensemble $P = \{$*Hassen, Yassine, Ahmed, Tarik, Omar, Saïd, Ali*$\}$. Une personne de P doit effectuer au moins une des activiés de A. On veut affecter les activités aux managers en respectant le fait que :

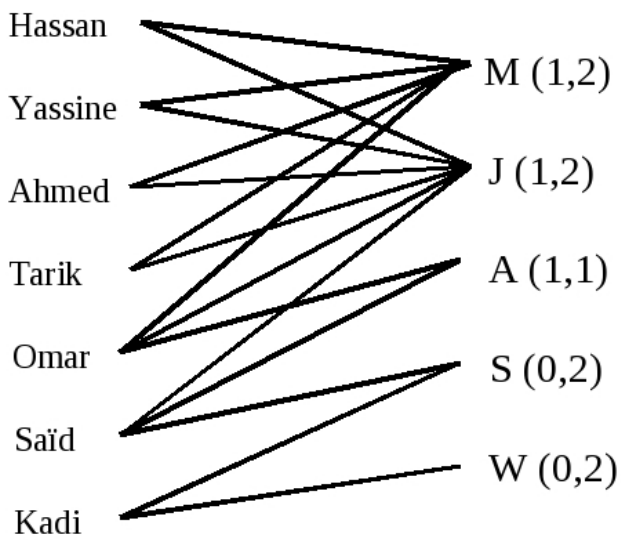
- Au moins une personne doit faire M, et au plus deux.
- Au moins une personne doit faire J, et au plus deux.
- Au moins une personne doit faire A, et pas plus d'une.
- Au plus deux personnes peuvent faire S.
- Au plus deux personnes peuvent faire W.
- *Hassan* est intéressé par M et J; *Yassine* par M et J; *Ahmed* par M et J; *Tarik* par M et J; *Omar* par M, J et A; *Sad* par J, A et S; *Ali* par S et W;

Exemple : emploi du temps - CSP

$$\mathcal{X} = \{H, Y, A, T, O, S, K\} \quad \mathcal{D} = \{D_H, D_Y, D_A, D_T, D_O, D_S, D_K\}$$
$$D_H = \{M, J\}, ..., D_A = \{S, W\} \qquad \mathcal{C} = \{C_1\}$$

Exemple : emploi du temps - CSP

$$\mathcal{X} = \{H, Y, A, T, O, S, K\} \quad \mathcal{D} = \{D_H, D_Y, D_A, D_T, D_O, D_S, D_K\}$$
$$D_H = \{M, J\}, ..., D_A = \{S, W\} \qquad \mathcal{C} = \{C_1\}$$
$$C_1 : \quad gcc(\{H, Y, A, T, O, S, K\}, \{M, J, A, S, W\}, [1, 1, 1, 0, 0], [2, 2, 1, 2, 2])$$



Exemple : ordonnancement

Soit un problème d'odonnancement :

- 7 tâches.
- Chaque tâche a une durée et une consommation de ressources.

tâche	t_1	t_2	t_3	t_4	t_5	t_6	t_7
durée	16	6	13	7	5	18	4
ressource	2	9	3	7	10	1	11

Ordonnancer les tâches sans dépasser la capacité en ressource 13 à n'importe quel instant. (Toute les tâches sont effectuées en moins de 30 minutes.)

Exemple : ordonnancement - CSP

$\mathcal{X} = LO \cup LE$
$LO = \{O_1, ..., O_7\}$
$LE = \{E_1, ..., E_7\}$
$D_{O_1} = ... = D_{O_7} = D_{E_1} = ... = D_{E_7} = [1, ..., 30]$
$C_1 : O_1 + 16 = E_1 \quad C_2 : O_2 + 6 = E_2 \quad C_3 : O_3 + 13 = E_3$
$C_4 : O_4 + 7 = E_4 \quad C_5 : O_5 + 5 = E_5 \quad C_6 : O_6 + 18 = E_6$
$C_7 : O_7 + 4 = E_7$
Soit $D = (16, 6, 13, 7, 5, 18, 4), R = (2, 9, 3, 7, 10, 1, 11), L = 13$
$C_8 : \forall i \in \mathbf{N}, \Sigma_{j|O_j \leq i \leq O_j + D_j - 1} R_j \leq 13$

Exemple : mélange chimique

Nous avons quatre réservoires : $R1, R2, R3$ et $R4$. Les deux premiers reçoivent trois produits de sources distinctes, puis leur contenu est combiné dans les deux autres afin de créer les mélanges désirés. La question est de déterminer la quantité de chaque produit à acheter afin de maximiser les profits.

$$
\begin{array}{ll}
 & x_1 \geq 0, \leq 100, x_2 \geq 0, \leq 100, \\
 & x_3 \geq 0, \leq 100, x_4 \geq 0, \leq 100, x_5 \geq 1, \leq 3. \\
\textit{minimize} & 120x_1 + 60x_2 + 10x_3 - 50x_4 - 50x_1x5 - 50x_2x_5 \\
\textit{subject to} & c_1 : 2.5x_1 + 0.5x_3 - x_1x_5 \geq 0 \\
 & c_2 : 1.5x_2 - 0.5x_4 - x_2x_5 \geq 0 \\
 & c_3 : x_1 + x_3 \leq 10 \\
 & c_4 : x_2 + x_4 \leq 20
\end{array}
$$

Exemple : PPC et test logiciel

- Test structurel :
  Trouver un jeu de cas de test dont le flot d'exécution passe par un point du programme !
- Démarches classiques :
  génération aléatoire !
  → un fruit du hasard !
- Avec la PPC :
  - Transformation du programme en un système de contraintes
  - Transformation du critère structurel en une contrainte
  - Résolution du système résultant
    - Pas de solution : du code mort
    - Des solutions : toutes sont des cas de test

```
int proc(int i)
  int j;
  j := 2;
  if (I ≤ 16) {
      j := j*I;
  }
  if (j > 8) {
      j := 0
  }
  return (j)
```

Exemple : PPC et test logiciel

```
int proc(int i)
   int j;
   j := 2;
   if (I ≤ 16) {
        j := j*i;
   }
   if (j > 8) {
        j := 0
   }
   return (j)
```

- Après un filtrage par *arc-consistance* : $i \in [5, 16]$
- Après une *énumération* : parmi les solutions $i=5$

$I_0 \in [0, 2^{16}-1]$

$J_1, J_2, J_3, J_4, J_5 \in [0, 2^{16}-1]$

$J_1 = 2$

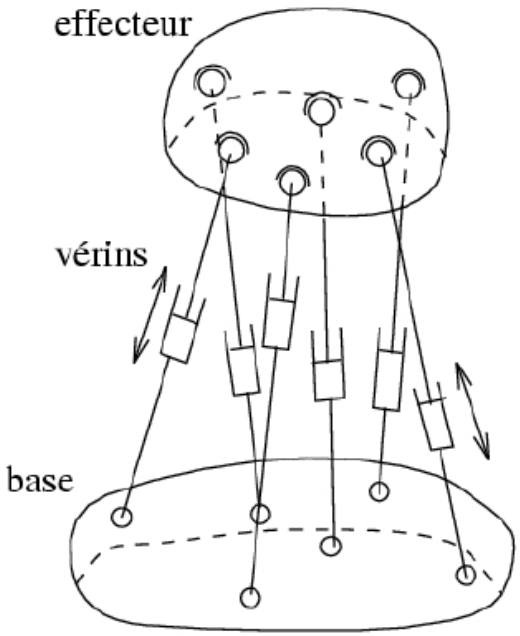
$ite(J_1 \leq 16, J_2 = J_1 * I_0 \wedge J_3 = J_2, J_3 = J_1)$

$ite(J_3 \leq 8, J_4 = 0 \wedge J_5 = J_4, J_5 = J_3)$

$RET = J_5$

$J3 > 8$

Exemple : PPC et robotique



Problem 1 (Gough-Stewart [11]).

$$
\begin{cases}
x_1^2 + y_1^2 + z_1^2 = 31;\ x_2^2 + y_2^2 + z_2^2 = 39;\ x_3^2 + y_3^2 + z_3^2 = 29; \\
x_1x_2 + y_1y_2 + z_1z_2 + 6x_1 - 6x_2 = 51; \\
x_1x_3 + y_1y_3 + z_1z_3 + 7x_1 - 2y_1 - 7x_3 + 2y_3 = 50; \\
x_2x_3 + y_2y_3 + z_1z_3 + x_2 - 2y_2 - x_3 + 2y_3 = 34; \\
-12x_1 + 15y_1 - 10x_2 - 25y_2 + 18x_3 + 18y_3 = -32; \\
-14x_1 + 35y_1 - 36x_2 - 45y_2 + 30x_3 + 18y_3 = 8; \\
2x_1 + 2y_1 - 14x_2 - 2y_2 + 8x_3 - y_3 = 20; \\
x_1 \in [-2.00, 5.57]; y_1 \in [-5.57, 2.70]; z_1 \in [0, 5.57] \\
x_2 \in [-6.25, 1.30]; y_2 \in [-6.25, 2.70]; z_2 \in [-2.00, 6.25] \\
x_3 \in [-5.39, 0.70]; y_3 \in [-5.39, 3.11]; z_3 \in [-3.61, 5.39]
\end{cases}
$$

Exemple : vérification et SAT

- Modélisation des problèmes de vérification sous forme de formules booléennes

$$(a_{11} \vee \ldots \vee a_{1l_1}) \wedge (a_{21} \vee \ldots \vee a_{2l_2}) \wedge \ldots \wedge (a_{n1} \vee \ldots \vee a_{nl_n})$$

- Résolution de ces formules avec les solveurs SAT

Résolution = Filtrage + Décomposition + Exploration

Résolution = Filtrage + Décomposition + Exploration

- Problèmes NP-complets, NP-difficiles, ... problèmes indécidables, ...

Résolution = Filtrage + Décomposition + Exploration

- Problèmes NP-complets, NP-difficiles, ... problèmes indécidables, ...
- ... difficulté dans le cas général, ... mais les instances peuvent être résolues "r a p i d e m e n t"
- Résolution = Filtrage + Décomposition + Exploration

Résolution = Filtrage + Décomposition + Exploration

- Problèmes NP-complets, NP-difficiles, ... problèmes indécidables, ...
- ... difficulté dans le cas général, ... mais les instances peuvent être résolues "r a p i d e m e n t"
- Résolution = Filtrage + Décomposition + Exploration
- Filtrage :
  - méthode de résolution disponible pour une contrainte C_i,
  - exploitation de cette méthode pour supprimer les valeurs ne participant pas aux solutions de C_i.

Résolution = Filtrage + Décomposition + Exploration

- Problèmes NP-complets, NP-difficiles, ... problèmes indécidables, ...
- ... difficulté dans le cas général, ... mais les instances peuvent être résolues "r a p i d e m e n t"
- Résolution = Filtrage + Décomposition + Exploration
- Filtrage :
  - méthode de résolution disponible pour une contrainte C_i,
  - exploitation de cette méthode pour supprimer les valeurs ne participant pas aux solutions de C_i.
- Décomposition : Séparation, branching, énumération, ...

Résolution = Filtrage + Décomposition + Exploration

- Problèmes NP-complets, NP-difficiles, ... problèmes indécidables, ...
- ... difficulté dans le cas général, ... mais les instances peuvent être résolues "r a p i d e m e n t"
- Résolution = Filtrage + Décomposition + Exploration
- Filtrage :
  - méthode de résolution disponible pour une contrainte C_i,
  - exploitation de cette méthode pour supprimer les valeurs ne participant pas aux solutions de C_i.
- Décomposition : Séparation, branching, énumération, ...
- Exploration : Parcours astucieux des noeuds de l'arbre de recherche

Définitions graphiques de base

hypergraphes des contraintes

A chaque CSP $\langle \mathcal{X}, \mathcal{D}, \mathcal{C} \rangle$ est associé un hypergraphe des contraintes obtenu en représentant chaque variable du CSP par un sommet et chaque contrainte $C_j(x_{j_1}, \ldots, x_{i_l})$ par une hyperarête entre $x_{j_1}, \ldots,$ et x_{i_l}.

hypergraphe de consistance

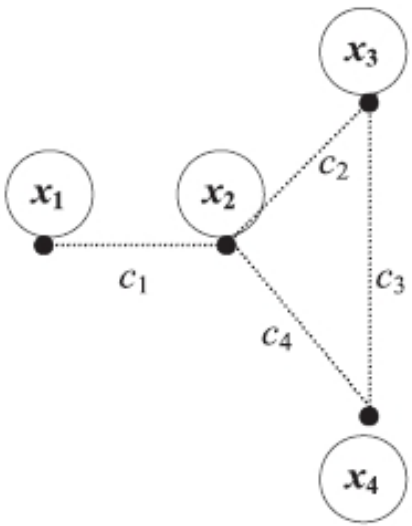
L'hypergraphe de consistance ou microstructure d'un CSP $\langle \mathcal{X}, \mathcal{D}, \mathcal{C} \rangle$ est l'hypergraphe dont les sommets sont le éléments de domaines d_i, et qui comporte une hyperarête $d_{i_1}, \ldots, d_{i_l}$ ssi le l-uplet est autorisé par la contrainte $C(x_{i_1}, \ldots, x_{i_l})$.

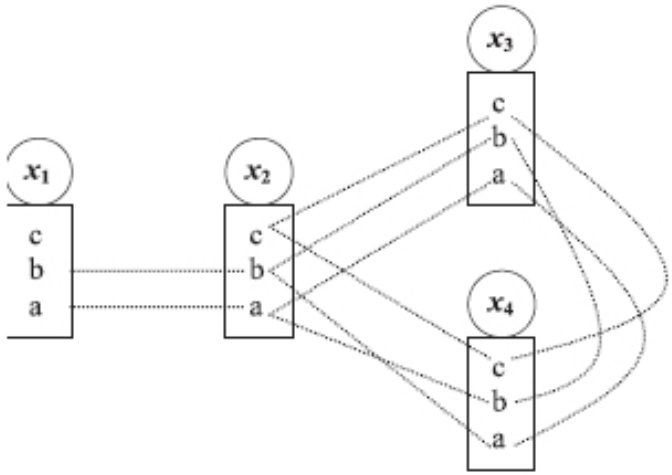
Définitions graphiques de base - exemples

Soit le CSP

- $\mathcal{X} = \{x_1, x_2, x_3, x_4\}$
- $\mathcal{D} = \{D_1, D_2, D_3, D_4\}$, avec $D_1 = D_2 = D_3 = D_4 = \{a, b, c\}$
- $\mathcal{C} = \{C_1(x_2, x_3), C_2(x_2, x_3), C_3(x_3, x_4), C_4(x_2, x_4)\}$ où :
  - $C_1(x_1, x_2) = \{(a, a), (b, b)\}$
  - $C_2(x_2, x_3) = C_3(x_3, x_4) = \{(a, a), (b, b), (c, c)\}$
  - $C_4(x_2, x_4) = \{(a, b), (b, a), (c, c)\}$

Définitions graphiques de base - exemples





Définitions de base

Instanciation partielle ou locale

Une instanciation partielle I de V est de la forme $\{x_{i_1} \rightarrow d_{i_1}, \ldots, x_{i_k} \rightarrow d_{i_k}\}$. I est un élément de $D_{i_1} \times \ldots \times D_{i_k}$.

Définitions de base

Instanciation partielle ou locale

Une instanciation partielle I de V est de la forme $\{x_{i_1} \rightarrow d_{i_1}, \ldots, x_{i_k} \rightarrow d_{i_k}\}$. I est un élément de $D_{i_1} \times \ldots \times D_{i_k}$.

Satisfiabilité d'une instanciation partielle

Une instanciation partielle $\langle d_{j_1}, \ldots, d_{j_l} \rangle$ vérifie une contrainte $C_j(x_{j_1}, \ldots, x_{j_l}\rangle)$ si et seulement si elle appartient à cette contrainte.

Définitions de base

Instanciation partielle ou locale

Une instanciation partielle I de V est de la forme $\{x_{i_1} \rightarrow d_{i_1}, \ldots, x_{i_k} \rightarrow d_{i_k}\}$. I est un élément de $D_{i_1} \times \ldots \times D_{i_k}$.

Satisfiabilité d'une instanciation partielle

Une instanciation partielle $\langle d_{j_1}, \ldots, d_{j_l} \rangle$ vérifie une contrainte $C_j(x_{j_1}, \ldots, x_{j_l}\rangle)$ si et seulement si elle appartient à cette contrainte.

Consistance locale d'une instanciation

Une instanciation partielle est dite localement consistante ssi elle satisfait toutes les contraintes se portant sur les variables de cette instanciation.

Définitions de base

Valeur compatible

Une valeur (x_i, d_i), avec $d_i \in D_i$, est dite compatible avec une instanciation localement consistante I, si $I \cup \{x_i \rightarrow d_i\}$ est localement consistante.

Définitions de base

Valeur compatible

Une valeur (x_i, d_i), avec $d_i \in D_i$, est dite compatible avec une instanciation localement consistante I, si $I \cup \{x_i \rightarrow d_i\}$ est localement consistante.

Instanciation globalement consistante

Une instanciation globalement consistante ou une solution d'un CSP $\langle \mathcal{X}, \mathcal{D}, \mathcal{C} \rangle$ est une instanciation complète de $\mathcal{X}$ localement consistante.

Définitions de base

Valeur compatible

Une valeur (x_i, d_i), avec $d_i \in D_i$, est dite compatible avec une instanciation localement consistante I, si $I \cup \{x_i \rightarrow d_i\}$ est localement consistante.

Instanciation globalement consistante

Une instanciation globalement consistante ou une solution d'un CSP $\langle \mathcal{X}, \mathcal{D}, \mathcal{C} \rangle$ est une instanciation complète de $\mathcal{X}$ localement consistante.

Instanciation partielle globalement consistante

Une instanciation partielle est globalement consistante ssi elle peut être étendue à une solution.

Définitions de base

Propriété

Si toutes les contraintes sont d'arité n, on aura $Sol(\mathcal{P}) = \mathcal{D} \cap C_1 \cap \ldots \cap C_m$.

Définitions de base

Propriété

Si toutes les contraintes sont d'arité n, on aura $Sol(\mathcal{P}) = \mathcal{D} \cap C_1 \cap \ldots \cap C_m$.

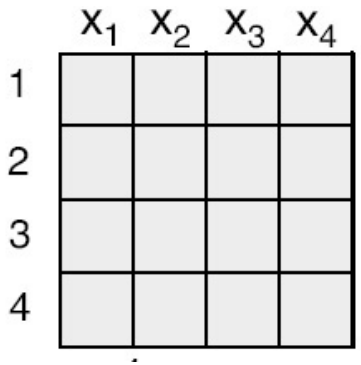
Equivalence de CSPs

Nous dirons que les deux CSP $\mathcal{P}_1$ et $\mathcal{P}_2$ sont équivalents si et seulement si $Sol(\mathcal{P}_1) = Sol(\mathcal{P}_2)$.

Résolution : exemple

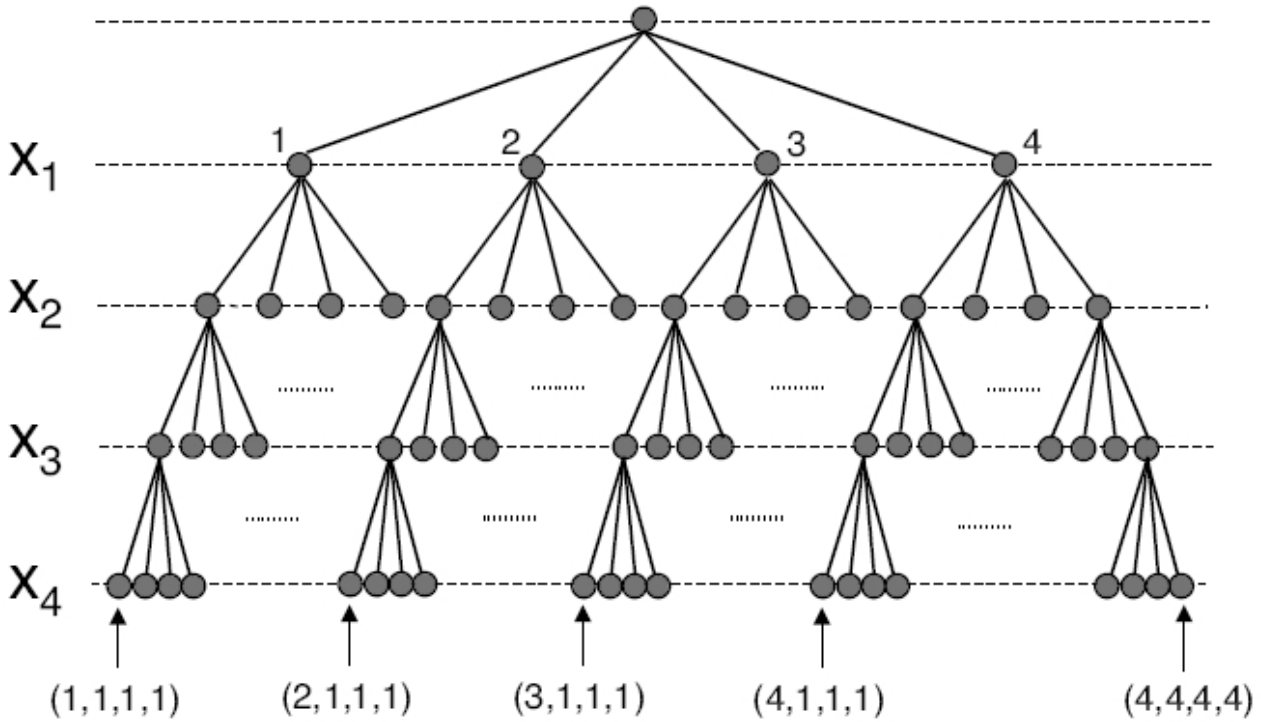
Soit le problème des quatres reines.

Soit $n = 4$ la taille de l'échiquier
$\mathcal{X} = \{x_1, ..., x_n\}$
$\mathcal{D} = \{D_1, ..., D_n\}, avec D_1 = ... D_N = [1, N]$
$\mathcal{C} = \{C_1, C_2\}$
$C_1 : x_1 \neq x_2 \neq ... \neq x_n$
$C_2 : |x_i - x_j| \neq |i - j|, 1 \leq i < j \leq n$



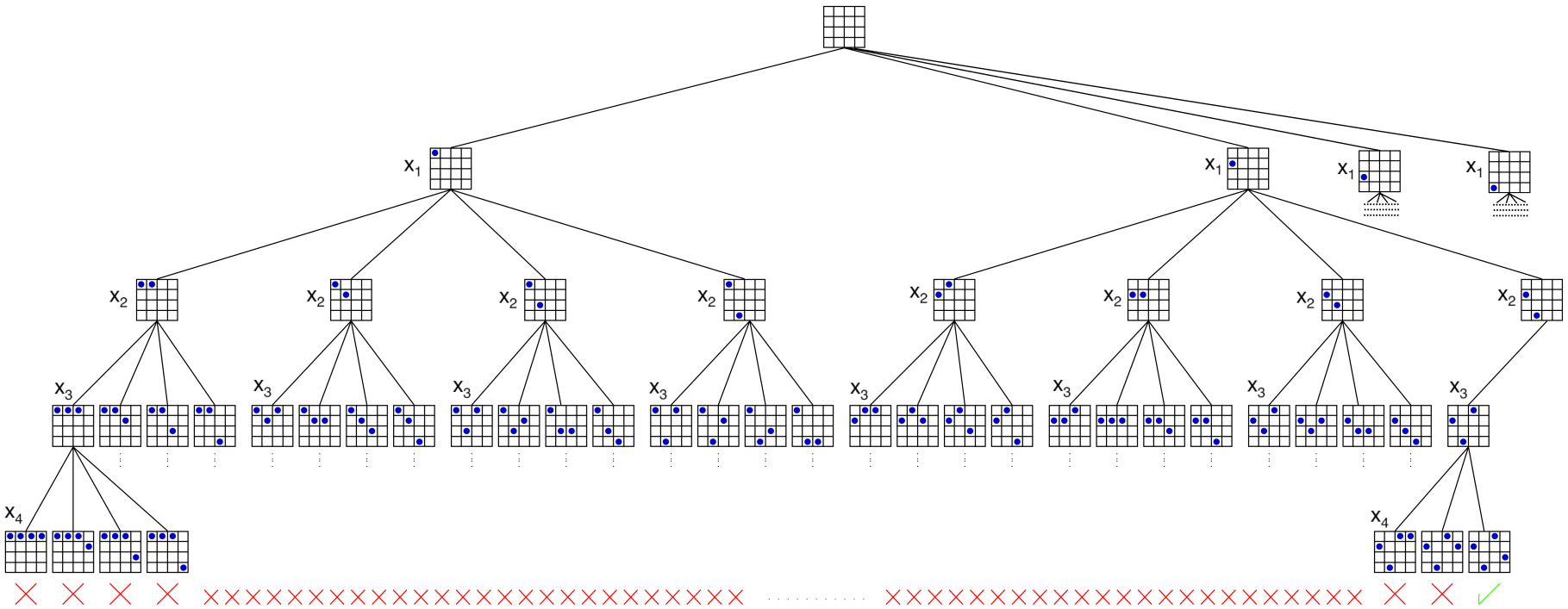
Résolution : espace de recherche

- Espace de recherche



Résolution : recherche naïve GT (generate and test)

- GT (generate and test) : générer toutes les combinaisons



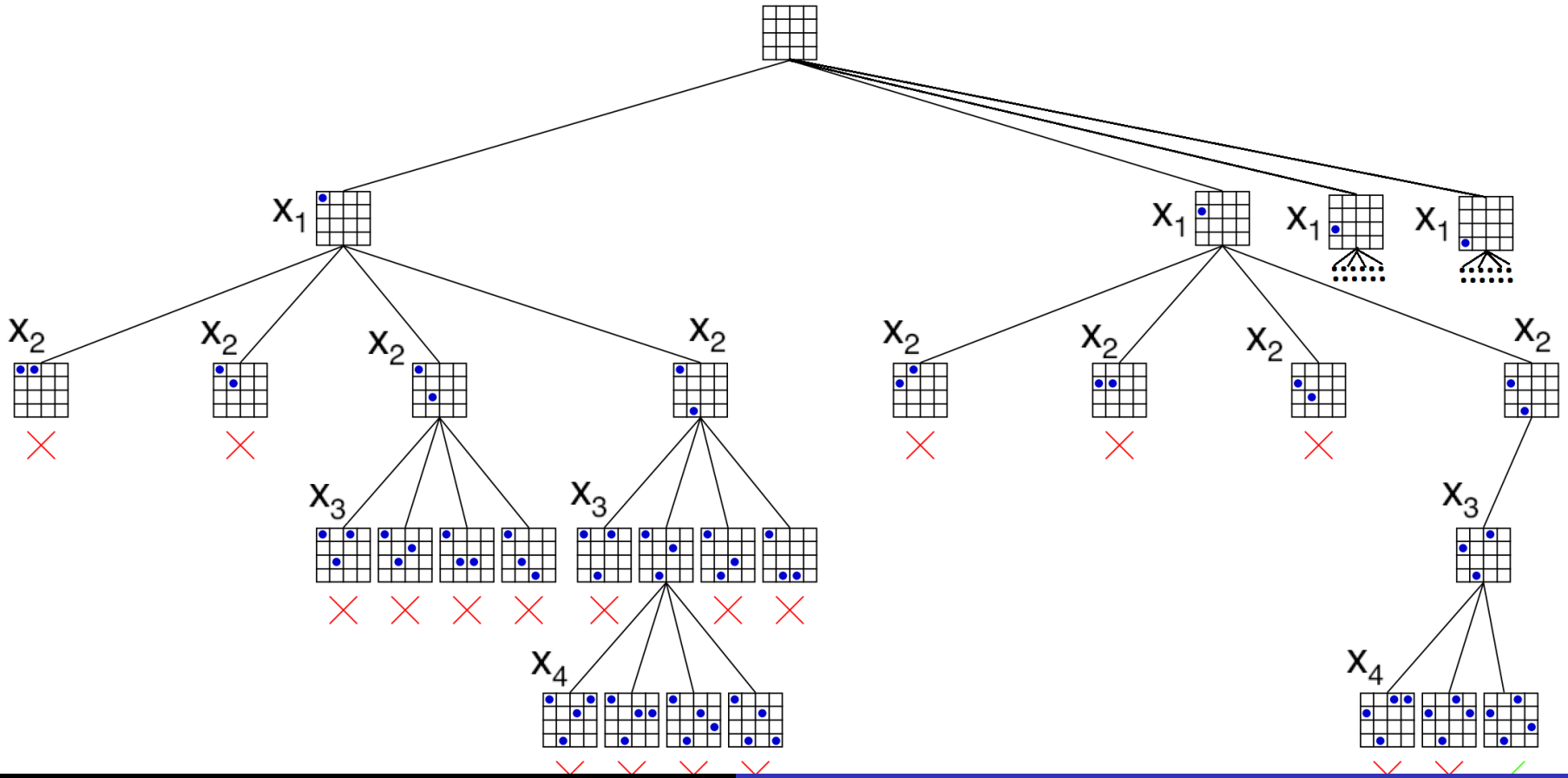
Résolution : recherche naïve GT (generate and test)

- Backtrack naïf : un algorithme sans filtrage, décomposition par énumération, et exploration par retour arrière.

1: Procédure Backtrack(in I)
2: $k := |I| + 1$
3: **for** $v_k \in D_k$ **do**
4: $\quad Iloc := I \cup \{x_k \rightarrow d_k\}$
5: $\quad$ **if** $k = n$ **then**
6: $\quad\quad$ **if** *Iloc* consistant **then**
7: $\quad\quad\quad$ afficher la solution *Iloc*
8: $\quad\quad$ **end if**
9: $\quad$ **else**
10: $\quad\quad$ Backtrack(*Iloc*)
11: $\quad$ **end if**
12: **end for**

Résolution : recherche incrémentale BT (Backtrack)

- BT (Backtrack) : vérifier incrémentalement les contraintes



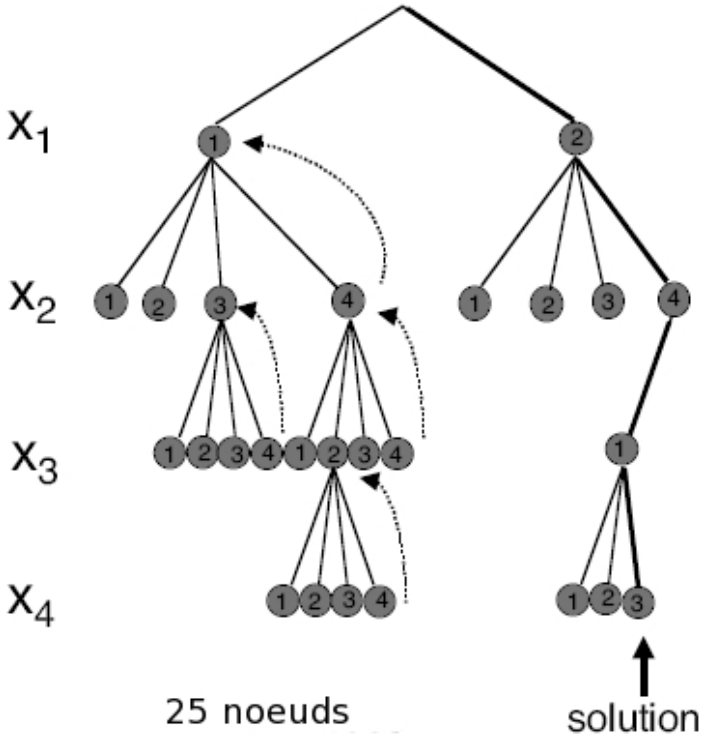
Résolution : recherche incrémentale BT (Backtrack)

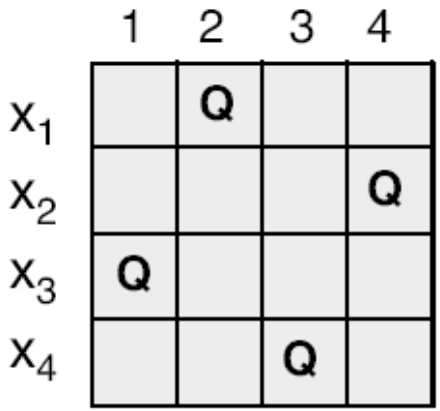
- Backtrack : un algorithme sans filtrage, décomposition par énumération, et exploration par retour arrière.

1: Procédure Backtrack(in I)
2: $k := |I| + 1$
3: **for** $v_k \in D_k$ **do**
4: $Iloc := I \cup \{x_k \rightarrow d_k\}$
5: **if** *Iloc* consistant **then**
6: **if** $k = n$ **then**
7: afficher la solution *Iloc*
8: **else**
9: Backtrack(*Iloc*)
10: **end if**
11: **end if**
12: **end for**

Résolution : l'algorithme simple Backtrack

- Comportement de Backtrack





Résolution : l'algorithme simple Backtrack

Résolution : l'algorithme simple Backtrack

- L'algorithme Backtrack représente le comportement le plus naïf en $O(d^n)$ d'un algorithme de résolution.
- Comment l'améliorer :
  - Filtrage : réduction des domaines

Résolution : l'algorithme simple Backtrack

- L'algorithme Backtrack représente le comportement le plus naïf en $O(d^n)$ d'un algorithme de résolution.
- Comment l'améliorer :
  - Filtrage : réduction des domaines
  - Décomposition : heuristiques de choix des variables

Résolution : l'algorithme simple Backtrack

- L'algorithme Backtrack représente le comportement le plus naïf en $O(d^n)$ d'un algorithme de résolution.
- Comment l'améliorer :
  - Filtrage : réduction des domaines
  - Décomposition : heuristiques de choix des variables
    - mindomain : Choisir la variable avec le plus petit domaine
    - degree : Choisir la variable impliquée dans le plus de contraintes
    - Combination : minimum des (mindomain/degree)

Résolution : l'algorithme simple Backtrack

- L'algorithme Backtrack représente le comportement le plus naïf en $O(d^n)$ d'un algorithme de résolution.
- Comment l'améliorer :
  - Filtrage : réduction des domaines
  - Décomposition : heuristiques de choix des variables
    - mindomain : Choisir la variable avec le plus petit domaine
    - degree : Choisir la variable impliquée dans le plus de contraintes
    - Combination : minimum des (mindomain/degree)
  - Exploration : remonter vers des noeuds, plus haut dans l'arbre de recherche, responsables de l'échec.

Filtrage - Exemple illustratif

Soit un CSP

$\mathcal{X} = \{x, y, z, t, r\}$

$D_x = D_y = \{0, 1\} \quad D_z = \{0, 1, 2\} \quad D_t = \{2, 3, 4\} \quad D_r = \{0, 1, 2, 3, 4\}$

$C_1 : x \neq y \neq z \quad C_2 : x^2 + t = z \quad C_3 : r^3 + x \geq 0$

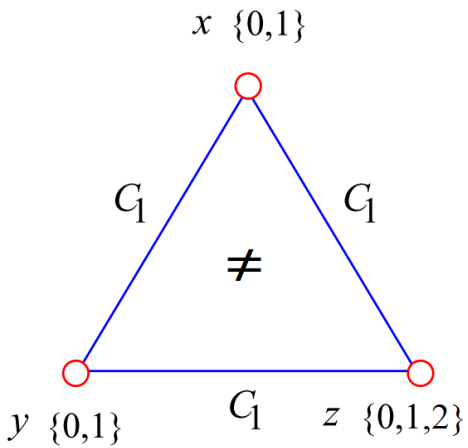
Filtrage - Exemple illustratif

Soit un CSP

$\mathcal{X} = \{x, y, z, t, r\}$

$D_x = D_y = \{0, 1\} \quad D_z = \{0, 1, 2\} \quad D_t = \{2, 3, 4\} \quad D_r = \{0, 1, 2, 3, 4\}$

$C_1 : x \neq y \neq z \quad C_2 : x^2 + t = z \quad C_3 : r^3 + x \geq 0$



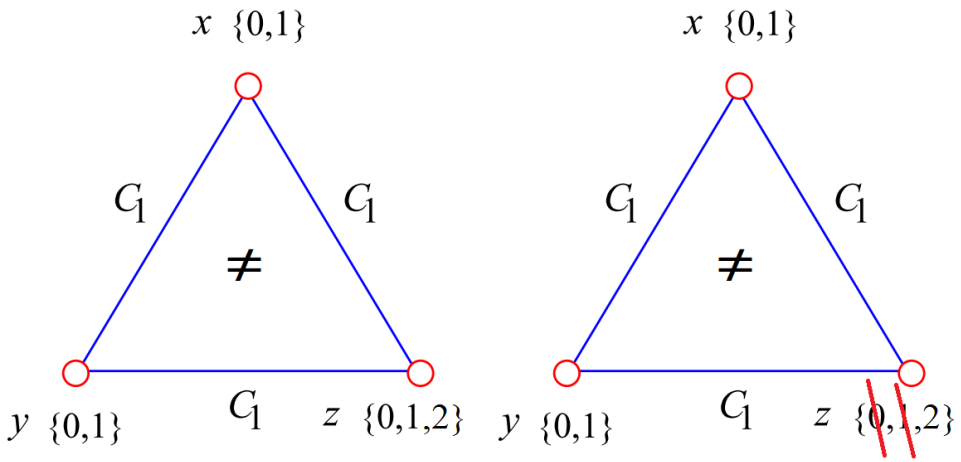
Filtrage - Exemple illustratif

Soit un CSP

$\mathcal{X} = \{x, y, z, t, r\}$

$D_x = D_y = \{0, 1\} \quad D_z = \{0, 1, 2\} \quad D_t = \{2, 3, 4\} \quad D_r = \{0, 1, 2, 3, 4\}$

$C_1 : x \neq y \neq z \quad C_2 : x^2 + t = z \quad C_3 : r^3 + x \geq 0$



Avec C_1, on peut déduire

$$D_z = \{2\}.$$

Filtrage - Exemple illustratif

Soit un CSP

$\mathcal{X} = \{x, y, z, t, r\}$
$D_x = D_y = \{0, 1\} \quad D_z = \{2\} \quad D_t = \{2, 3, 4\} \quad D_r = \{0, 1, 2, 3, 4\}$
$C_1 : x \neq y \neq z \quad C_2 : x^2 + t = z \quad C_3 : r^3 + x \geq 0$

C_2 prend la forme $x^2 + t = 2$, d'où :

$$D_x = \{0\} \quad D_t = \{2\}.$$

Filtrage - Exemple illustratif

Soit un CSP

$\mathcal{X} = \{x, y, z, t, r\}$
$D_x = D_y = \{0, 1\} \quad D_z = \{2\} \quad D_t = \{2, 3, 4\} \quad D_r = \{0, 1, 2, 3, 4\}$
$C_1 : x \neq y \neq z \quad C_2 : x^2 + t = z \quad C_3 : r^3 + x \geq 0$

C_2 prend la forme $x^2 + t = 2$, d'où :

$$D_x = \{0\} \quad D_t = \{2\}.$$

Puis, on revient à C_1 qui prend désormais la forme $0 \neq y \neq 2$, d'où :

$$D_y = \{1\}.$$

Filtrage - Exemple illustratif

Soit un CSP

$\mathcal{X} = \{x, y, z, t, r\}$
$D_x = D_y = \{0, 1\} \quad D_z = \{2\} \quad D_t = \{2, 3, 4\} \quad D_r = \{0, 1, 2, 3, 4\}$
$C_1 : x \neq y \neq z \quad C_2 : x^2 + t = z \quad C_3 : r^3 + x \geq 0$

C_2 prend la forme $x^2 + t = 2$, d'où :

$$D_x = \{0\} \quad D_t = \{2\}.$$

Puis, on revient à C_1 qui prend désormais la forme $0 \neq y \neq 2$, d'où :

$$D_y = \{1\}.$$

Enfin :

$$D_x = \{0\}, D_y = \{1\}, D_z = \{2\} \quad D_t = \{2\} \quad D_r = \{0, 1, 2, 3, 4\}.$$

Filtrage - AC3

1: $L := \{C_1, ..., C_m\}$; % toutes les contraintes

Filtrage - AC3

1: $L := \{C_1, ..., C_m\}$; % toutes les contraintes
2: **while** L est non vide **do**
3: Choisir et supprimer dans L une contrainte $C_j(x_{j_1}, \ldots, x_{j_l})$;

Filtrage - AC3

1: $L := \{C_1, ..., C_m\}$; % toutes les contraintes
2: **while** L est non vide **do**
3: Choisir et supprimer dans L une contrainte $C_j(x_{j_1}, \ldots, x_{j_l})$;
4: **for** $x_k \in \{x_{j_1}, \ldots, x_{j_l}\}$ **do**
5: Réduire le domaine de x_k avec la contrainte C_j

Filtrage - AC3

1: $L := \{C_1, ..., C_m\}$; % toutes les contraintes
2: **while** L est non vide **do**
3: Choisir et supprimer dans L une contrainte $C_j(x_{j_1}, \ldots, x_{j_l})$;
4: **for** $x_k \in \{x_{j_1}, \ldots, x_{j_l}\}$ **do**
5: Réduire le domaine de x_k avec la contrainte C_j
6: Si réduction alors $L := L \cup \{C_p | x_k \in var(C_p)\}$;
7: **end for**
8: **end while**

Filtrage - Exemple illustratif

Soit un CSP

$$\begin{aligned} \mathcal{X} = & \quad \{x, y\} \\ & \quad D_x = [10, 20] \\ & \quad D_y = [5, 15] \\ C_1 : & \quad x < y \end{aligned}$$

Filtrage - Exemple illustratif

Soit un CSP

$$\begin{aligned} \mathcal{X} = & \quad \{x, y\} \\ & \quad D_x = [10, 20] \\ & \quad D_y = [5, 15] \\ C_1 : & \quad x < y \end{aligned}$$

Avec C_1, on peut déduire

$$D_x = [10, 14], D_y = [11, 15].$$

Filtrage et *arc*-consistance

Etant donné un CSP $\langle \mathcal{X}, \mathcal{D}, \mathcal{C} \rangle$:

- $C_i(x_{i_1}, \ldots, x_{i_l}) \subseteq D_{i_1} \times \ldots \times D_{i_l}$

Filtrage et *arc*-consistance

Etant donné un CSP $\langle \mathcal{X}, \mathcal{D}, \mathcal{C} \rangle$:

- $C_i(x_{i_1}, \ldots, x_{i_l}) \subseteq D_{i_1} \times \ldots \times D_{i_l}$
- Un tuple $(d_{i_1}, \ldots, d_{i_l}) \in C_i$ est dit Solution de C_i.

Filtrage et *arc*-consistance

Etant donné un CSP $\langle \mathcal{X}, \mathcal{D}, \mathcal{C} \rangle$:

- $C_i(x_{i_1}, \ldots, x_{i_l}) \subseteq D_{i_1} \times \ldots \times D_{i_l}$
- Un tuple $(d_{i_1}, \ldots, d_{i_l}) \in C_i$ est dit Solution de C_i.
- On dit aussi que ce tuple satisfait C_i.

Filtrage et *arc*-consistance

Etant donné un CSP $\langle \mathcal{X}, \mathcal{D}, \mathcal{C} \rangle$:

- $C_i(x_{i_1}, \ldots, x_{i_l}) \subseteq D_{i_1} \times \ldots \times D_{i_l}$
- Un tuple $(d_{i_1}, \ldots, d_{i_l}) \in C_i$ est dit Solution de C_i.
- On dit aussi que ce tuple satisfait C_i.

arc-consistance

Une valeur $v \in D_{x_i}$ est *arc*-consistante si et seulement si elle est localement consistante. Sinon, on dit que v est non-*arc*-consistante.

Filtrage et *arc*-consistance : formellement

Etant donné un CSP $\langle \mathcal{X}, \mathcal{D}, \mathcal{C} \rangle$:

- Ce CSP est *arc*-consistant si et seulement si pour toute variable x_i et toute valeur $d_i \in D_{x_i}$, d_i est *arc*-consistante.
- d_i est *arc*-consistante si et seulement si

Filtrage et *arc*-consistance : formellement

Etant donné un CSP $\langle \mathcal{X}, \mathcal{D}, \mathcal{C} \rangle$:

- Ce CSP est *arc*-consistant si et seulement si pour toute variable x_i et toute valeur $d_i \in D_{x_i}$, d_i est *arc*-consistante.
- d_i est *arc*-consistante si et seulement si

$$\begin{array}{l} \forall C_j(x_{j_1}, \ldots, x_{j_k} \ldots, x_{j_l}), \\ \quad x_i = x_{j_k}, \\ \quad \exists d_{i_1}, \ldots, \exists d_{j_{k-1}}, \exists d_{j_{k+1}}, \ldots, \exists d_{j_l} \\ \quad (d_{i_1} \in D_{i_1}, \ldots, d_{j_{k-1}} \in D_{j_{k-1}}, d_{j_{k+1}} \in D_{j_{k+1}}, \ldots, d_{j_l} \in D_{j_l}) \\ (d_{i_1}, \ldots, d_i, \ldots, d_{j_l}) \in C_j \end{array}$$

Filtrage : propagation

- Tant que il existe des valeurs non-*arc*-consistantes, les supprimer, et continuer à détecter de nouvelles valeurs inconsistantes.
- Soit $\pi_{j,k}(\mathcal{D})$ l'ensemble des valeurs de la variable x_k qui sont *arc*-consistantes relativement à la contrainte C_j :

$$\begin{array}{l}\pi_{j,i}(\mathcal{D}) = \\ \{d_i | d_i \in D_i, \\ \quad x_i = x_{j_k}, \\ \quad \exists d_{i_1}, \ldots, \exists d_{j_{k-1}}, \exists d_{j_{k+1}}, \ldots, \exists d_{j_l} \\ \quad (d_{i_1} \in D_{i_1}, \ldots, d_{j_{k-1}} \in D_{j_{k-1}}, d_{j_{k+1}} \in D_{j_{k+1}}, \ldots, d_{j_l} \in D_{j_l}) \\ (d_{i_1}, \ldots, d_i, \ldots, d_{j_l}) \in C_j\}.\end{array}$$

Filtrage : Propagation - AC3

1: $L := \mathcal{C}$;
2: **while** L est non vide **do**
3: Choisir et supprimer dans L une contrainte $C_j(x_{j_1}, \ldots, x_{j_l})$;
4: **for** $x_k \in \{x_{j_1}, \ldots, x_{j_l}\}$ **do**
5: **if** $\pi_{j,k}(\mathcal{D}) \neq D_k$ **then**
6: $D_k := \pi_{j,k}(\mathcal{D})$;
7: $L := L \cup \{C_p | x_k \in var(C_p)\}$;
8: **end if**
9: **end for**
10: **end while**

Contraintes globales

- Le calcul de la projection $\pi_{j,k}(\mathcal{D}) \neq D_k$ est de complexité exponentielle d^n où d est la cardinalité *max* d'un domaine.
- Si c_i est d'arité 2, dite contrainte binaire, $\pi_{j,k}(\mathcal{D})$ est polynomiale.
- Comment faire avec les contraintes n-aires ?

Contraintes globales

- Le calcul de la projection $\pi_{j,k}(\mathcal{D}) \neq D_k$ est de complexité exponentielle d^n où d est la cardinalité max d'un domaine.
- Si c_i est d'arité 2, dite contrainte binaire, $\pi_{j,k}(\mathcal{D})$ est polynomiale.
- Comment faire avec les contraintes n-aires ?
  - La projection est exponentielle dans le cas général.

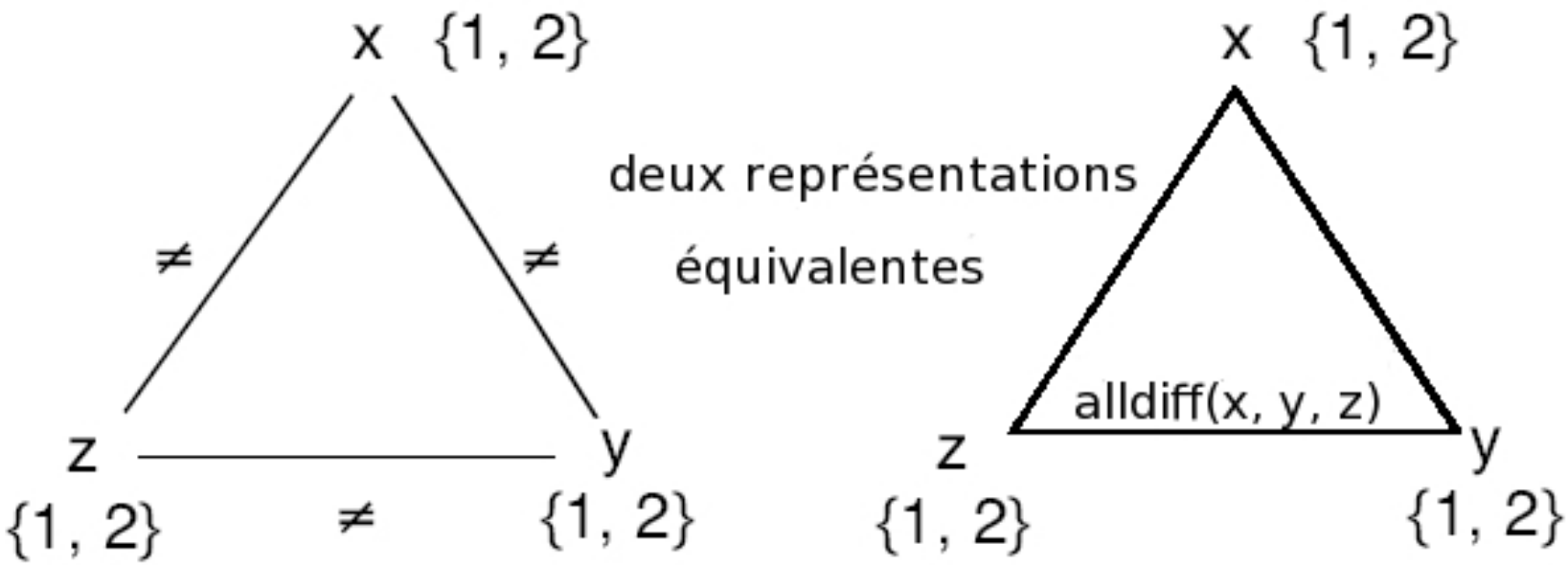
Contraintes globales

- Le calcul de la projection $\pi_{j,k}(\mathcal{D}) \neq D_k$ est de complexité exponentielle d^n où d est la cardinalité max d'un domaine.
- Si c_i est d'arité 2, dite contrainte binaire, $\pi_{j,k}(\mathcal{D})$ est polynomiale.
- Comment faire avec les contraintes n-aires ?
  - La projection est exponentielle dans le cas général.
  - La binarisation des contraintes n-aires ne filtre pas assez !

Contraintes globales

- Le calcul de la projection $\pi_{j,k}(\mathcal{D}) \neq D_k$ est de complexité exponentielle d^n où d est la cardinalité max d'un domaine.
- Si c_i est d'arité 2, dite contrainte binaire, $\pi_{j,k}(\mathcal{D})$ est polynomiale.
- Comment faire avec les contraintes n-aires ?
  - La projection est exponentielle dans le cas général.
  - La binarisation des contraintes n-aires ne filtre pas assez !
  - Idée : Proposer une projection dédiée à chaque contrainte ! Terrain d'intégration des techniques RO

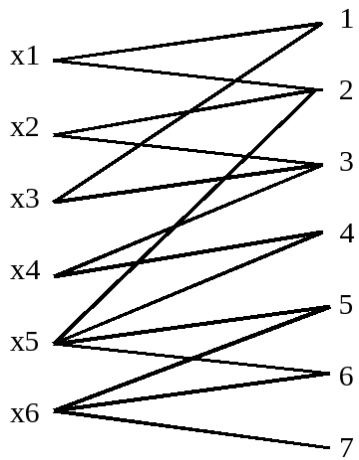
Consistance partielle : faiblesse binarisation



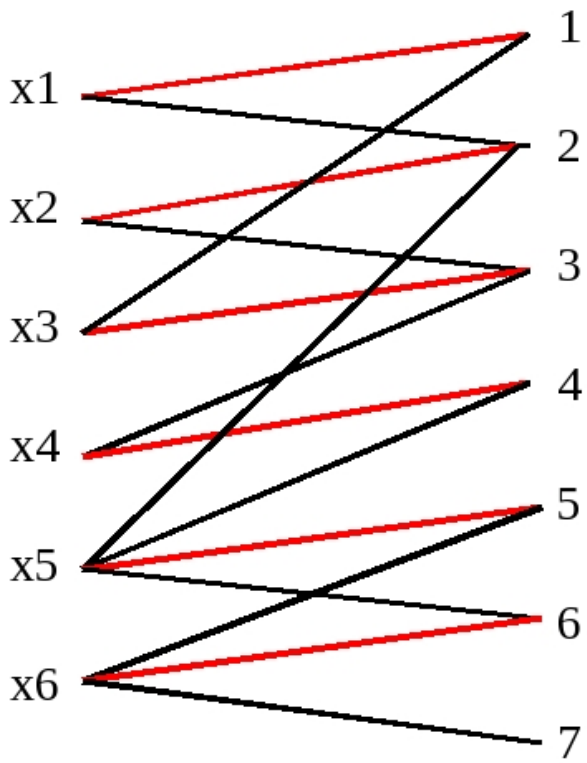
Consistance partielle : contraintes globales - alldiff

La contrainte *alldif* impose que les valeurs prises par un ensemble de variables soient différentes deux à deux.
Soit un CSP

$$\begin{aligned} \mathcal{X} = & \quad \{x_1, ..., x_6\} \\ & \quad D_{x_1} = \{1, 2\}, ..., D_{x_6} = \{5, 6, 7\} \\ C_1 : & \quad x_1 \neq x_2 \neq x_3 \neq x_4 \neq x_5 \neq x_6 \end{aligned}$$



Contraintes globales - alldiff - couplage

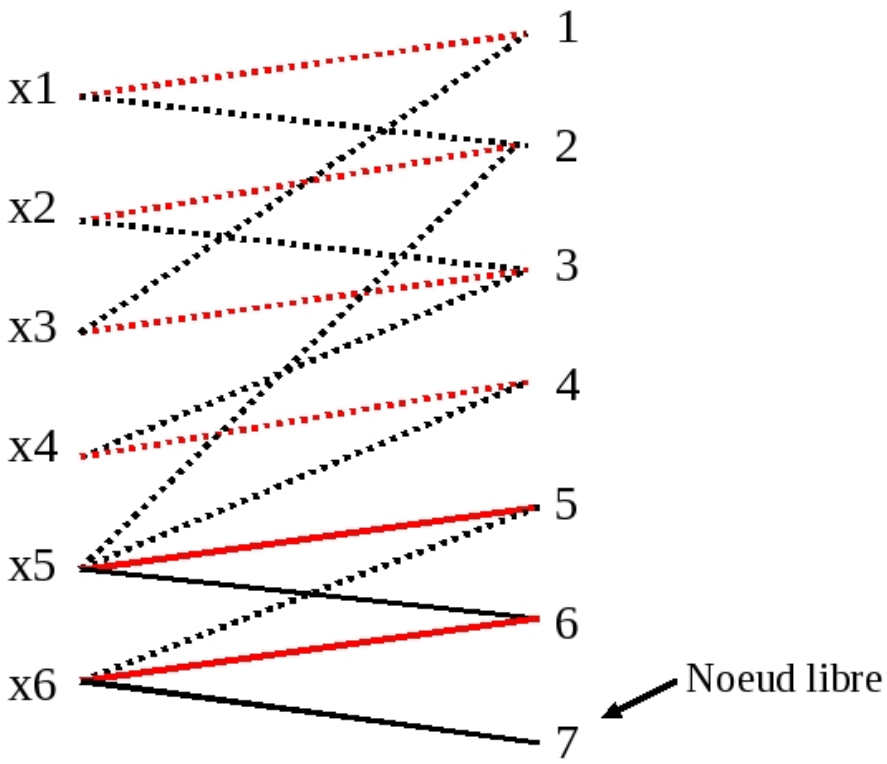


Contraintes globales - alldiff - exploitation du couplage

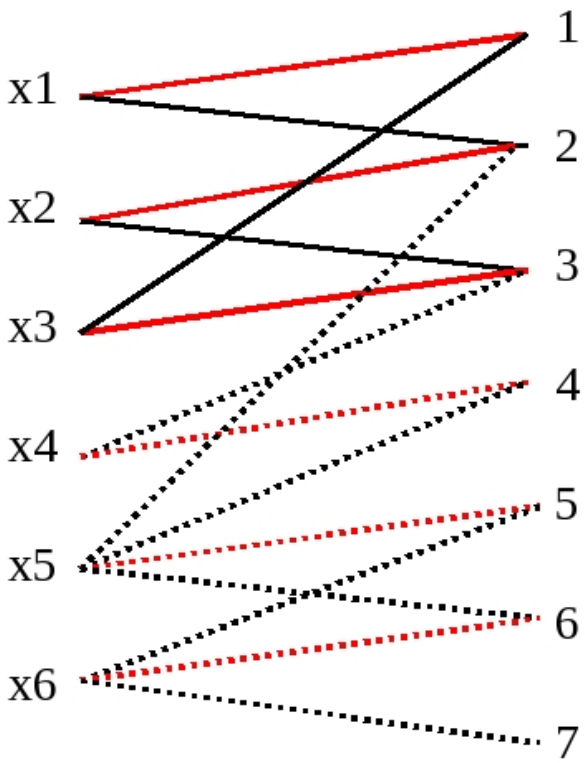
Propriété de Berge

Une arête appartient à des couplages maximum mais non à tous si et seulement si pour un couplage maximum arbitraire M, cette arête appartient soit à une chaîne alternée paire qui commence à un noeud libre, soit à un cycle alterné pair.

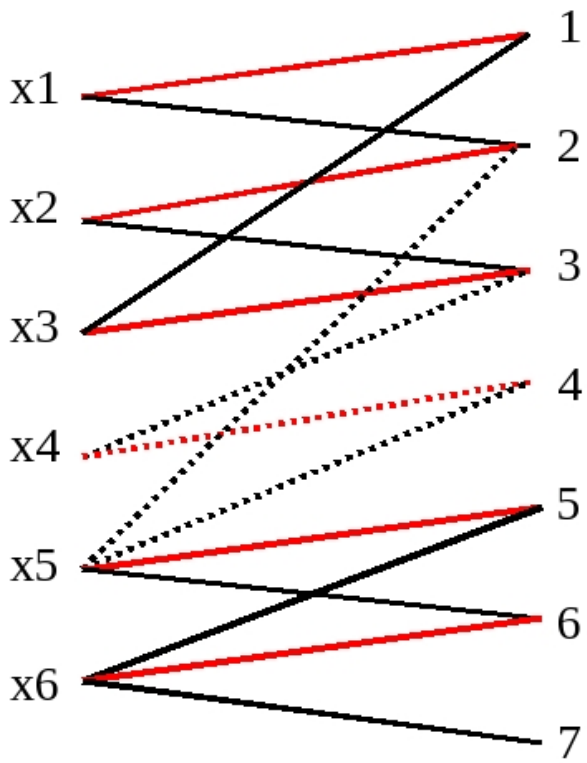
Contraintes globales - alldiff - chaîne alternée paire



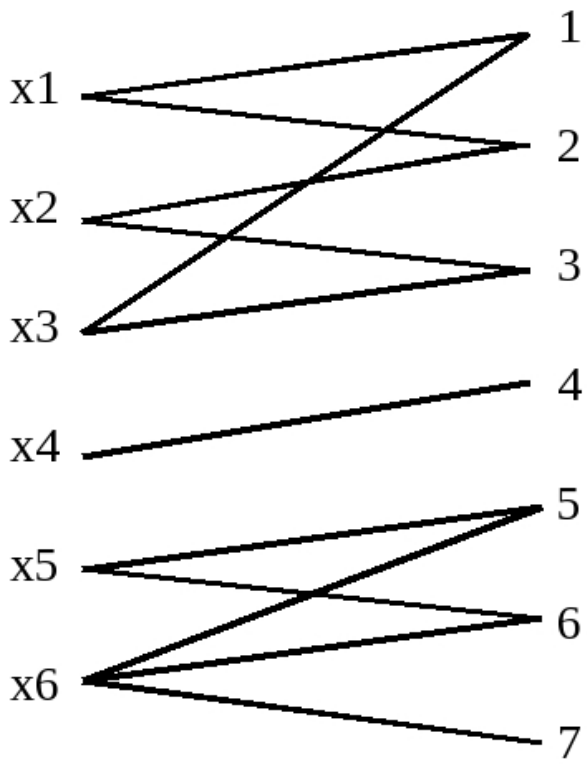
Contraintes globales - alldiff - cycle alterné pair



Contraintes globales - alldiff - arêtes consistantes



Contraintes globales - alldiff



Contraintes globales - alldiff - algorithme

- Calculer les composantes fort. connexes dans le graphe G_o :

Contraintes globales - alldiff - algorithme

- Calculer les composantes fort. connexes dans le graphe G_o :
  - Etant donné le graphe biparti (X, Y, E) avec un couplage M qui couvre X, $G_o(X, Y, Succ)$ obtenu en orientant les arêtes :
    - $\forall x \in X, Succ(x) = \{y \in Y | (x, y) \in M\}$
    - $\forall y \in Y, Succ(y) = \{x \in X | (x, y) \in E - M\}$

Contraintes globales - alldiff - algorithme

- Calculer les composantes fort. connexes dans le graphe G_o :
  - Etant donné le graphe biparti (X, Y, E) avec un couplage M qui couvre X, $G_o(X, Y, Succ)$ obtenu en orientant les arêtes :
    - $\forall x \in X, Succ(x) = \{y \in Y | (x, y) \in M\}$
    - $\forall y \in Y, Succ(y) = \{x \in X | (x, y) \in E - M\}$
  - Tout cycle dans G_o est un cycle alterné pair dans G, et inversement.
  - Tout chemin dans G_o qui commence par un noeud libre correspond à un chaîne alternée pair dans G qui commence par un noeud libre. L'inverse est vrai.

Contraintes globales - alldiff - algorithme

- Calculer les composantes fort. connexes dans le graphe G_o :
  - Etant donné le graphe biparti (X, Y, E) avec un couplage M qui couvre X, $G_o(X, Y, Succ)$ obtenu en orientant les arêtes :
    - $\forall x \in X, Succ(x) = \{y \in Y | (x, y) \in M\}$
    - $\forall y \in Y, Succ(y) = \{x \in X | (x, y) \in E - M\}$
  - Tout cycle dans G_o est un cycle alterné pair dans G, et inversement.
  - Tout chemin dans G_o qui commence par un noeud libre correspond à un chaîne alternée pair dans G qui commence par un noeud libre. L'inverse est vrai.
- Supprimer toutes les arêtes qui n'appartiennent pas à une composante fortement connexe, et n'appartiennent pas aux chemins de G_0 qui commencent par un noeud libre.

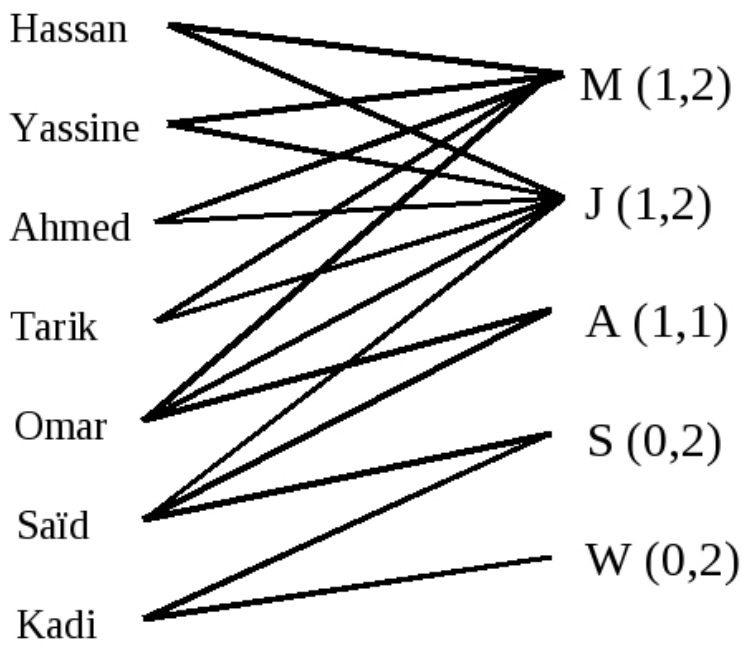
Contraintes globales - alldiff - algorithme

- Calculer les composantes fort. connexes dans le graphe G_o :
  - Etant donné le graphe biparti (X, Y, E) avec un couplage M qui couvre X, $G_o(X, Y, Succ)$ obtenu en orientant les arêtes :
    - $\forall x \in X, Succ(x) = \{y \in Y | (x, y) \in M\}$
    - $\forall y \in Y, Succ(y) = \{x \in X | (x, y) \in E - M\}$
  - Tout cycle dans G_o est un cycle alterné pair dans G, et inversement.
  - Tout chemin dans G_o qui commence par un noeud libre correspond à un chaîne alternée pair dans G qui commence par un noeud libre. L'inverse est vrai.
- Supprimer toutes les arêtes qui n'appartiennent pas à une composante fortement connexe, et n'appartiennent pas aux chemins de G_0 qui commencent par un noeud libre.
- Complexité : $O(n^{1/2}m) = O(n^{3/2}d)$
- Complexité de l'algorithme incrémental : $O(knd)$

Contraintes globales - gcc

Soit $C = gcc(\{x_{i_1}, ..., x_{i_l}\}, V, l, u)$ la contrainte de cardinalité :

$$C = \{\tau | \tau \in D_{x_{i_1}} \times ... \times D_{x_{i_l}} \text{ et } \forall a_i \in V : l_i \leq \#(a_i, \tau) \leq u_i\}$$



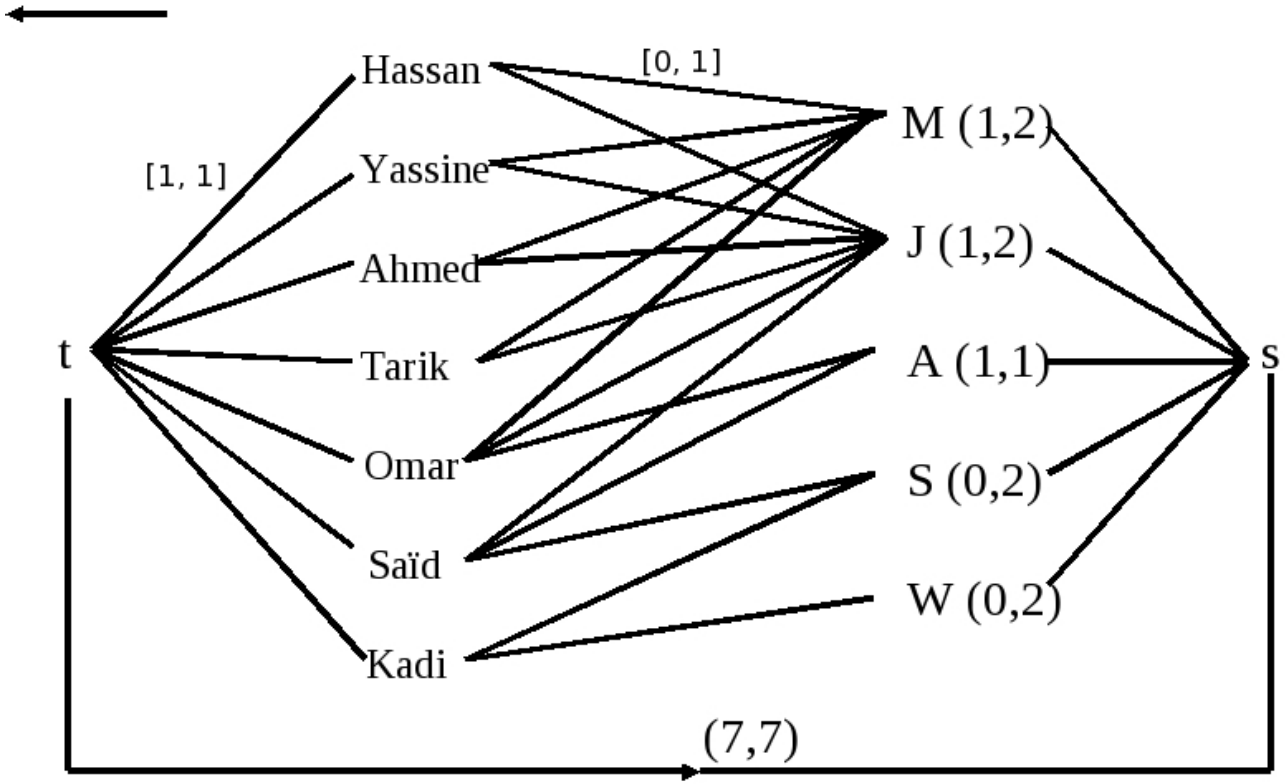
Contraintes globales - gcc - réseau valué

Définition - Réseau valué [Régin 1996]

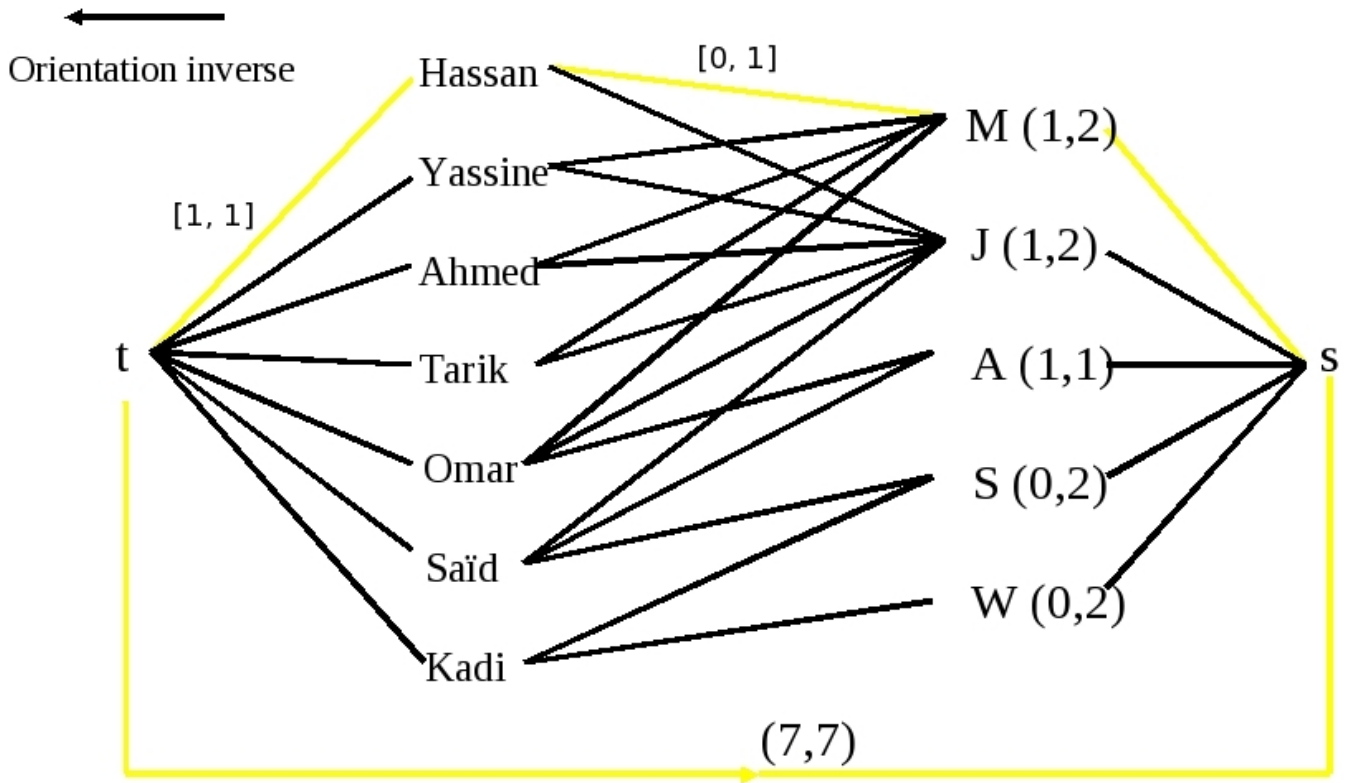
Soit $C = gcc(X, V, l, u)$ une GCC ; le **réseau de valeurs** *de C est le graphe biparti orienté $N(C)$ munit de bornes inférieures et supérieures de capacité sur chaque arc. L'ensemble des nœuds de $N(C)$ est formé par l'ensemble X, l'ensemble V et deux nœuds supplémentaires s et t. Les arcs de $N(C)$ sont définis de la façon suivante :*

- *il existe un arc entre une valeur a de V et une variable x de X si et seulement si $a \in D(x)$. Pour chacun de ces arcs (a, x) on a $l_{ax} = 0$ et $u_{ax} = 1$.*
- *il existe un arc entre s et toutes les valeurs a_i de V. Pour chaque arc (s, a_i) on a $l_{sa_i} = l_i$ et $u_{sa_i} = u_i$.*
- *il existe un arc entre toutes les variables x de X et t. Pour chaque arc (x, t) on a $l_{xt} = 1$, $u_{xt} = 1$.*
- *il existe un arc (t, s) avec $l_{ts} = u_{ts} = |X(C)|$.*

Contraintes globales - gcc - réseau valué



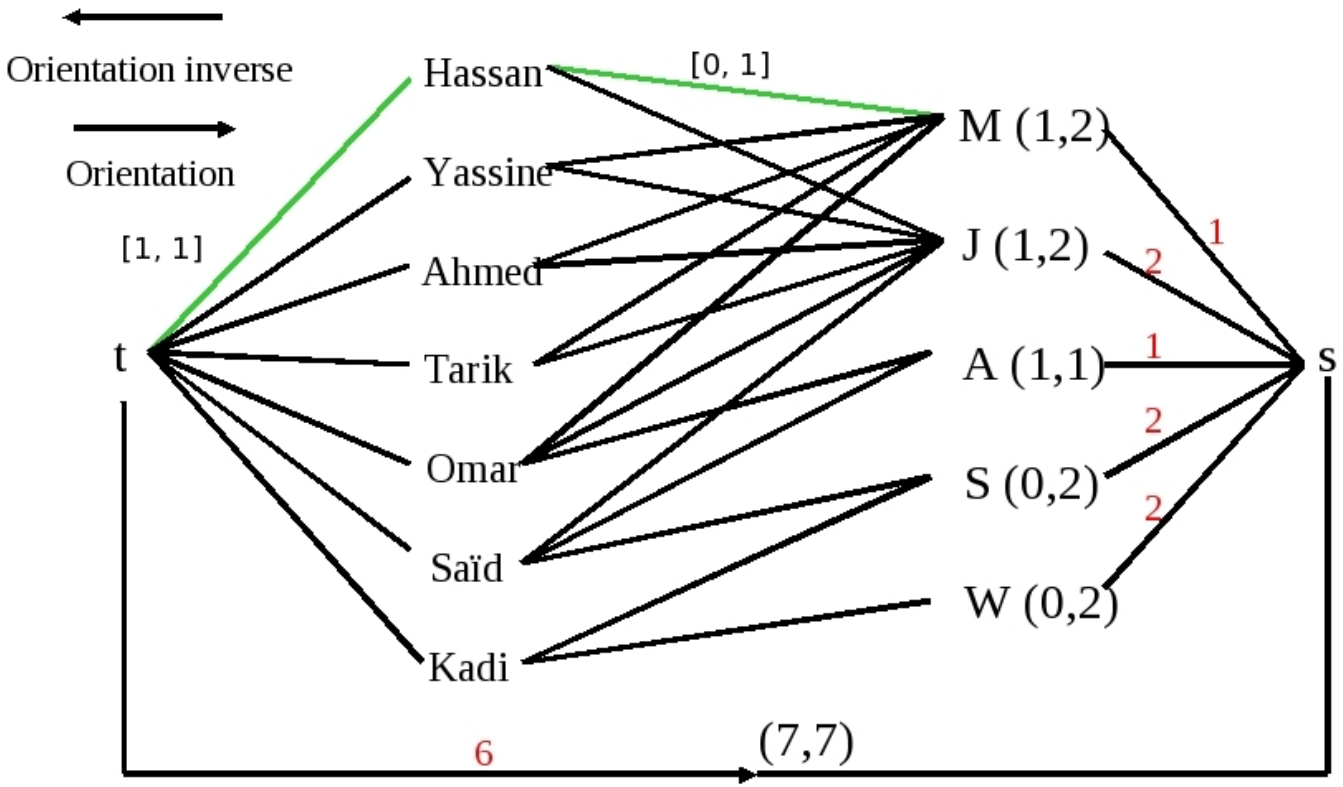
Contraintes globales - gcc - flot réalisable



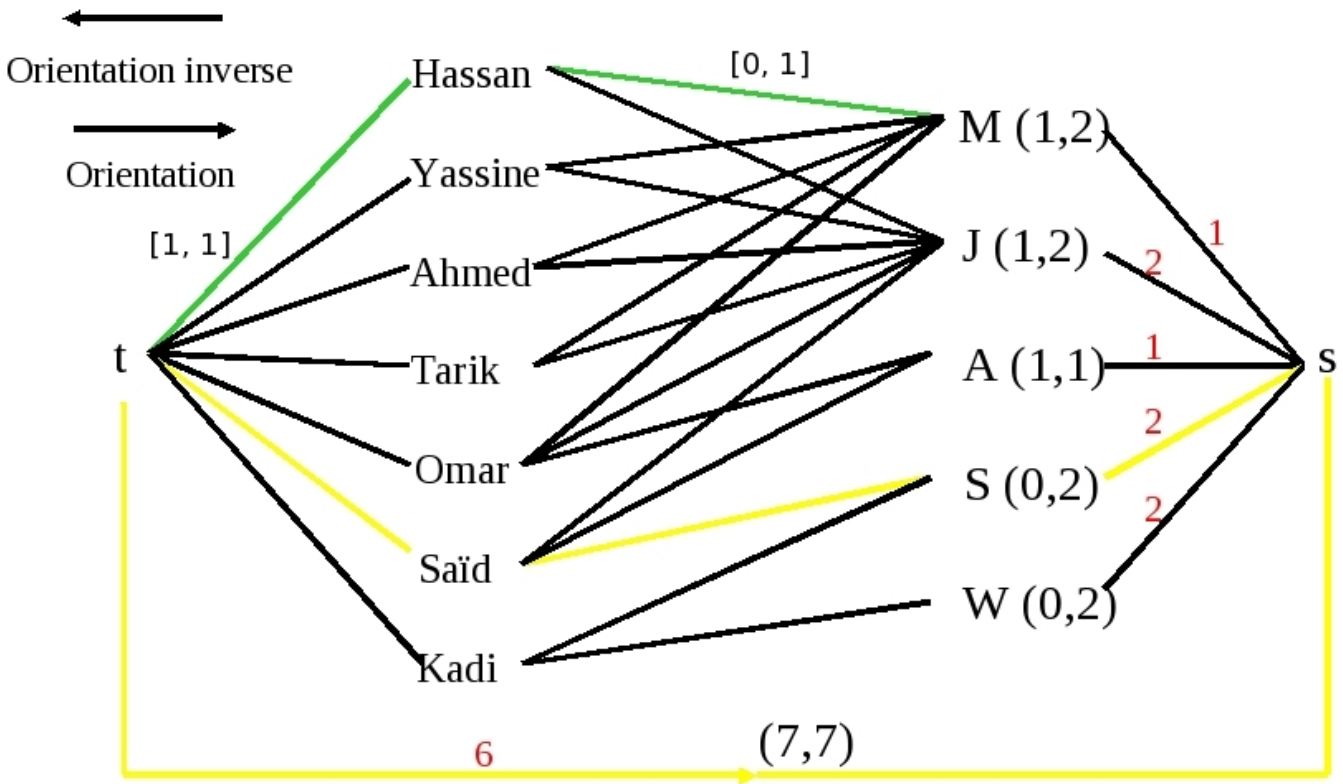
Contraintes globales - gcc - amélioration du flot courant

- Les augmentations du flot sont calculées dans le graphe résiduel.
- Le graphe résiduel n'a pas de bornes inférieures.
- Le graphe résiduel d'un flot f, noté $R(f)$, est le graphe orienté ayant le même ensemble de noeuds que G. L'ensemble des arcs de $R(f)$ est défini. $\forall (i,j) \in U(G)$:
  - $f_{ij} < u_{ij} \equiv (i,j) \in U(R(f))$ et sa borne supérieure de capacité est $r_{ij} = u_{ij} - f_{ij}$.
  - $f_{ij} > l_{ij} \equiv (j,i) \in U(R(f))$ et sa borne supérieure de capacité est $r_{ji} = f_{ij} - l_{ij}$.
  - Toutes les bornes inférieures de capacité sont nulles.

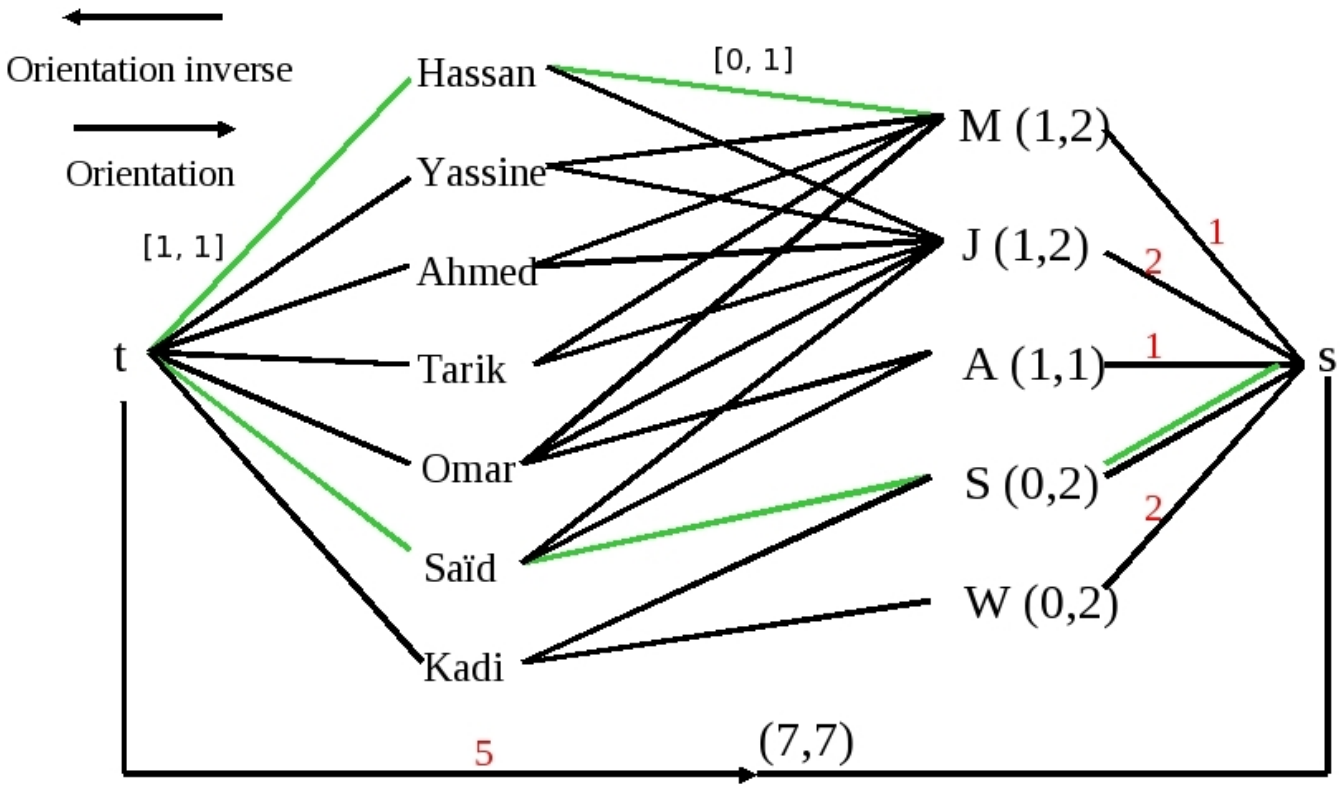
Contraintes globales - gcc - graphe résiduel



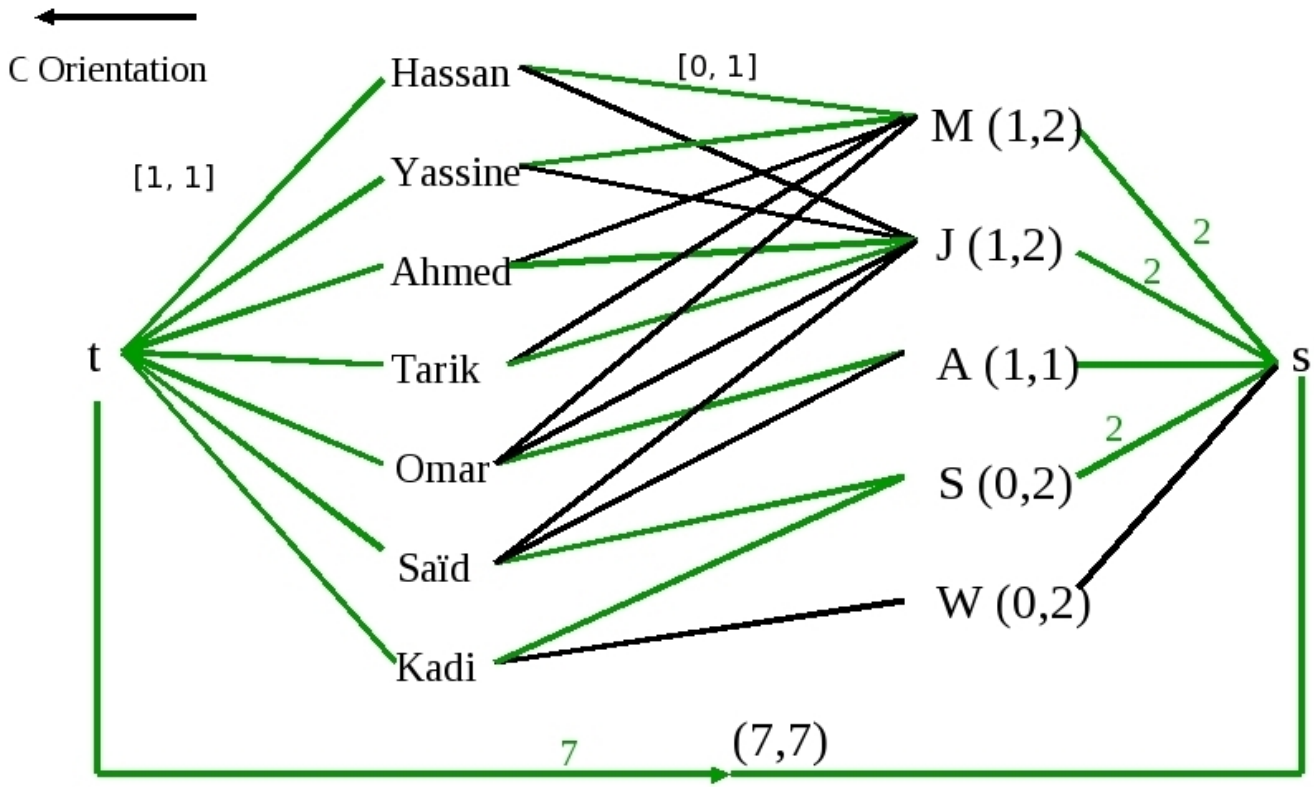
Contraintes globales - gcc - graphe résiduel



Contraintes globales - gcc - graphe résiduel



Contraintes globales - gcc - une solution



Contraintes globales - gcc - propriétés

Proposition - Consistance des domaines

([Régin, 1996]) *Soient C une GCC consistante et f un flot compatible de $N(C)$. Une valeur a d'une variable x n'est pas consistante avec C si et seulement si $f_{ax} = 0$ et a et x appartiennent à des composantes fortement connexes différentes de $R(f)$.*

Contraintes globales - gcc - algorithme

- Trouver un flot réalisable

Contraintes globales - gcc - algorithme

- Trouver un flot réalisable
- Calculer les composantes fortement connexes

Contraintes globales - gcc - algorithme

- Trouver un flot réalisable
- Calculer les composantes fortement connexes
- Supprimer tout arc ayant un flux nul et n'appartenant pas à une composante fortement connexe

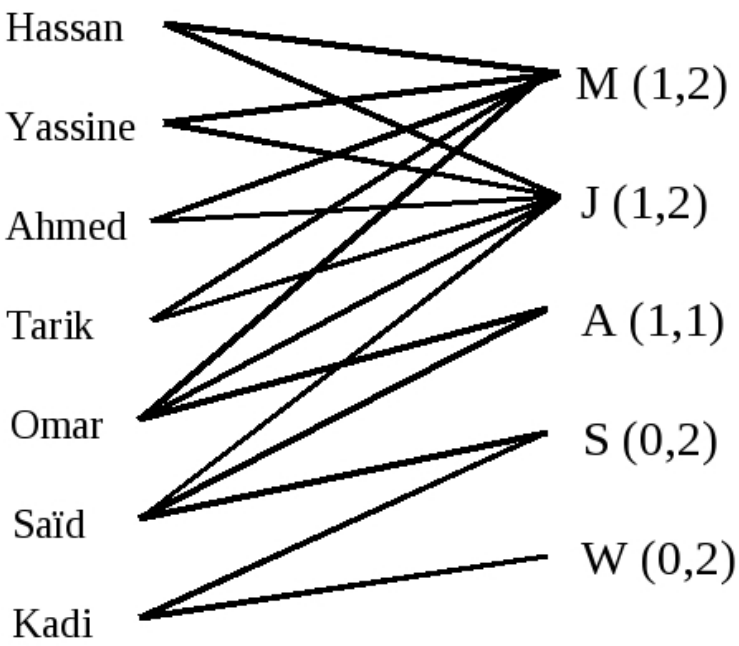
Contraintes globales - gcc - algorithme

- Trouver un flot réalisable
- Calculer les composantes fortement connexes
- Supprimer tout arc ayant un flux nul et n'appartenant pas à une composante fortement connexe
- Algorithme linéaire établissant l'*arc*-consistance

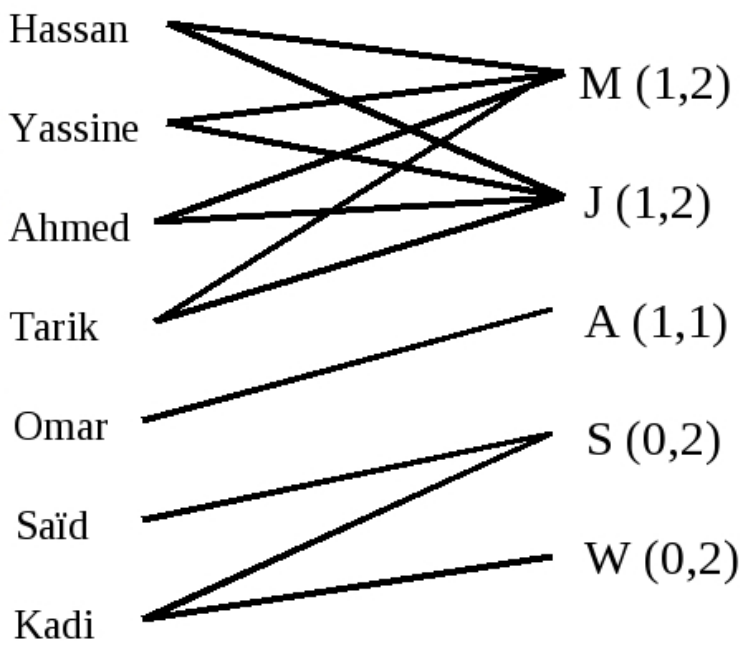
Contraintes globales - gcc - algorithme

- Trouver un flot réalisable
- Calculer les composantes fortement connexes
- Supprimer tout arc ayant un flux nul et n'appartenant pas à une composante fortement connexe
- Algorithme linéaire établissant l'*arc*-consistance
- Fonctionne rapidement car les arcs ont des capacités 0 ou 1

Contraintes globales - gcc - exemple



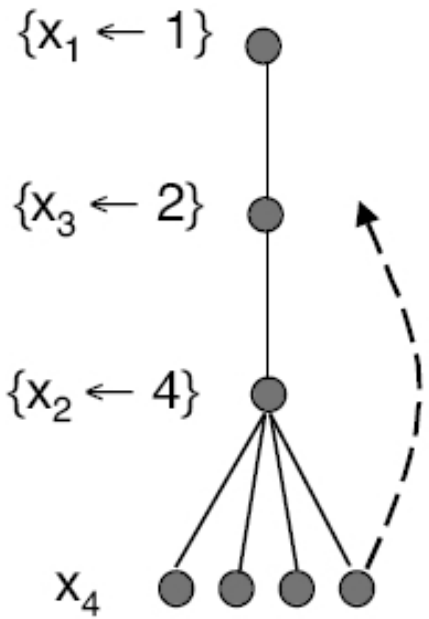
Contraintes globales - gcc - exemple filtré

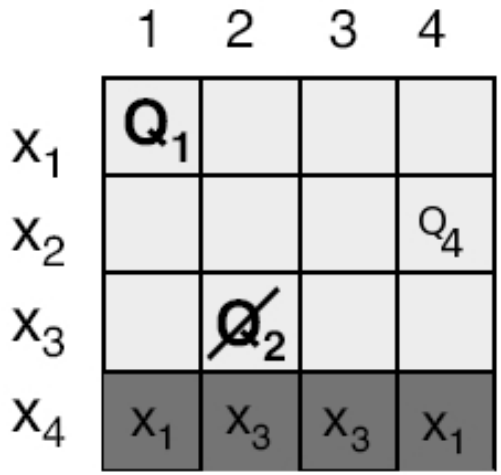


Exploration : Thrashing

- Thrashing : le même échec peut être redécouvert plusieurs fois, ... un nombre exponentiel de fois, ...
- Idée : remonter plus haut dans l'arbre de recherche
  - Backjumping
  - Dynamic Backtracking

Exploration : Thrashing - Illustration





Résolution - Mécanismes pour backtrack intelligent

- Soit une instanciation partielle P localement consistante. $\overline{P}$ est l'ensemble des variables de P. Une explication éliminante au niveau d'une variable x_i est une paire (d_i, S) où $d_i \in D_i$ et $S \subseteq \overline{P}$. On veut exprimer le fait que la variable x_i ne peut pas prendre comme valeur d_i à cause des valeurs déjà assignées à $\overline{P}$.

Résolution - Mécanismes pour backtrack intelligent

- Soit une instanciation partielle P localement consistante. $\overline{P}$ est l'ensemble des variables de P. Une explication éliminante au niveau d'une variable x_i est une paire (d_i, S) où $d_i \in D_i$ et $S \subseteq \overline{P}$. On veut exprimer le fait que la variable x_i ne peut pas prendre comme valeur d_i à cause des valeurs déjà assignées à $\overline{P}$.
- Un mécanisme d'explication ϵ d'un CSP est une fonction qui a deux arguments : une instanciation parielle P, et une variable $x_i \notin P(1)$. Cette fonction retourne un ensemble (peut être vide) d'explications éliminantes.

Résolution - Mécanismes pour backtrack intelligent

- Soit une instanciation partielle P localement consistante. $\overline{P}$ est l'ensemble des variables de P. Une explication éliminante au niveau d'une variable x_i est une paire (d_i, S) où $d_i \in D_i$ et $S \subseteq \overline{P}$. On veut exprimer le fait que la variable x_i ne peut pas prendre comme valeur d_i à cause des valeurs déjà assignées à $\overline{P}$.
- Un mécanisme d'explication ϵ d'un CSP est une fonction qui a deux arguments : une instanciation parielle P, et une variable $x_i \notin P(1)$. Cette fonction retourne un ensemble (peut être vide) d'explications éliminantes.
- Soit E un ensemble d'explications éminantes. $\hat{E}$ dénote les valeurs à éliminer. $\hat{\epsilon}(P, x_i)$ est l'ensemble des valeurs à éliminer.

Résolution - Mécanismes pour backtrack intelligent

Trois hypothèses sont essentielles pour le bien fondé d'un mécanisme d'explication ϵ :

Résolution - Mécanismes pour backtrack intelligent

Trois hypothèses sont essentielles pour le bien fondé d'un mécanisme d'explication ϵ :

- Correction : pour toute valeur $v_i \notin \hat{\epsilon}(P, x_i)$, toute contrainte contenant des variables de $\overline{P} \cup \{x_i\}$ est satisfaite par $P \cup \{(x_i, v_i)\}$.

Résolution - Mécanismes pour backtrack intelligent

Trois hypothèses sont essentielles pour le bien fondé d'un mécanisme d'explication ϵ :

- Correction : pour toute valeur $v_i \notin \hat{\epsilon}(P, x_i)$, toute contrainte contenant des variables de $\overline{P} \cup \{x_i\}$ est satisfaite par $P \cup \{(x_i, v_i)\}$.
- Complétude : Soit P une instanciation partielle localement consistante. Soit P' une instanciation partielle qui étend P avec $(v, E) \in \epsilon(P', x_i)$ alors $E \cap (\overline{P'} - \overline{P}) \neq \emptyset$.

Résolution - Mécanismes pour backtrack intelligent

Trois hypothèses sont essentielles pour le bien fondé d'un mécanisme d'explication ϵ :

- Correction : pour toute valeur $v_i \notin \hat{\epsilon}(P, x_i)$, toute contrainte contenant des variables de $\overline{P} \cup \{x_i\}$ est satisfaite par $P \cup \{(x_i, v_i)\}$.
- Complétude : Soit P une instanciation partielle localement consistante. Soit P' une instanciation partielle qui étend P avec $(v, E) \in \epsilon(P', x_i)$ alors $E \cap (\overline{P'} - \overline{P}) \neq \emptyset$.
- Précision : Etant données une solution partielle P, une variable x_i, et une valeur v_i, alors il existe au plus un seul élément de la forme $(v_i, E) \in \epsilon(P, x_i)$.

Recherche en profondeur [Ginsberg, JAIR]

Algorithm (Depth-first search) *Given as inputs a constraint-satisfaction problem and an elimination mechanism* ϵ*:*

1. *Set* $P = \emptyset$*.* P *is a partial solution to the* CSP*. Set* $E_i = \emptyset$ *for each* $i \in I$*;* E_i *is the set of values that have been eliminated for the variable* i*.*

2. *If* $\overline{P} = I$*, so that* P *assigns a value to every element in* I*, it is a solution to the original problem. Return it. Otherwise, select a variable* $i \in I - \overline{P}$*. Set* $E_i = \widehat{\epsilon}(P, i)$*, the values that have been eliminated as possible choices for* i*.*

3. *Set* $S = V_i - E_i$*, the set of remaining possibilities for* i*. If* S *is nonempty, choose an element* $v \in S$*. Add* (i, v) *to* P*, thereby setting* i*'s value to* v*, and return to step 2.*

4. *If* S *is empty, let* (j, v_j) *be the last entry in* P*; if there is no such entry, return failure. Remove* (j, v_j) *from* P*, add* v_j *to* E_j*, set* $i = j$ *and return to step 3.*

Backjumping [Ginsberg, JAIR]

Algorithm (Backjumping) *Given as inputs a constraint-satisfaction problem and an elimination mechanism ϵ:*

1. *Set $P = E_i = \emptyset$ for each $i \in I$.*
2. *If $\overline{P} = I$, return P. Otherwise, select a variable $i \in I - \overline{P}$. Set $E_i = \epsilon(P, i)$.*
3. *Set $S = V_i - \widehat{E}_i$. If S is nonempty, choose an element $v \in S$. Add (i, v) to P and return to step 2.*
4. *If S is empty, we must have $\widehat{E}_i = V_i$. Let E be the set of all variables appearing in the explanations for each eliminated value.*
5. *If $E = \emptyset$, return failure. Otherwise, let (j, v_j) be the last entry in P such that $j \in E$. Remove from P this entry and any entry following it. Add $(v_j, E \cap \overline{P})$ to E_j, set $i = j$ and return to step 3.*

Dynamic backtracking [Ginsberg, JAIR]

Algorithm (Dynamic backtracking) *Given as inputs a constraint-satisfaction problem and an elimination mechanism* ϵ*:*

1. *Set* $P = E_i = \emptyset$ *for each* $i \in I$.

2. *If* $\overline{P} = I$, *return* P. *Otherwise, select a variable* $i \in I - \overline{P}$. *Set* $E_i = E_i \cup \epsilon(P, i)$.

3. *Set* $S = V_i - \widehat{E}_i$. *If* S *is nonempty, choose an element* $v \in S$. *Add* (i, v) *to* P *and return to step 2.*

4. *If* S *is empty, we must have* $\widehat{E}_i = V_i$; *let* E *be the set of all variables appearing in the explanations for each eliminated value.*

5. *If* $E = \emptyset$, *return failure. Otherwise, let* (j, v_j) *be the last entry in* P *that binds a variable appearing in* E. *Remove* (j, v_j) *from* P *and, for each variable* k *assigned a value after* j, *remove from* E_k *any eliminating explanation that involves* j. *Add* $(v_j, E \cap \overline{P})$ *to* E_j *and return to step 2.*

Exemple ... [Ginsberg, JAIR]

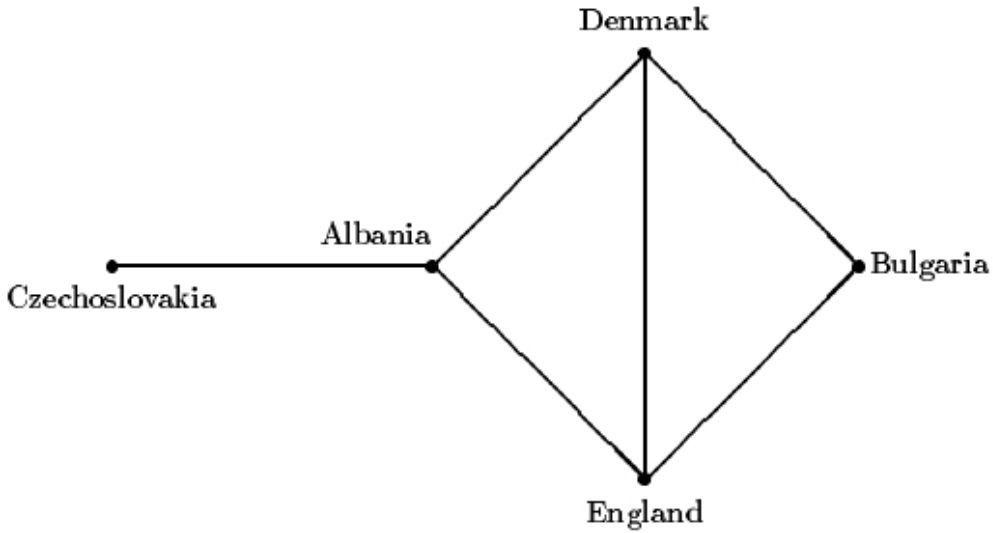


Figure 1: A small map-coloring problem

country	color	red	yellow	blue
Albania	red			
Bulgaria	yellow			
Czechoslovakia	blue	A		
Denmark				
England				

Résolution - Le problème SAT

Motivation: le problème

We consider the general constrained optimization problem

$$\mathcal{P} \equiv \begin{cases} \text{minimize} & h(x) \\ \text{subject to} & c_i(x) = 0, \ i \in \mathcal{E} \\ & c_i(x) \geq 0, \ i \in \mathcal{I} \\ & \underline{x_i} \leq x_i \leq \overline{x_i}, x_i \in \mathcal{X} \end{cases}$$

Critical issue: Efficient and safe computation of the global minimum

Mésaventures du calcul flottant

Soit l'expression

$$333.75 \times y^6 + x^2 \times (11 \times x^2 \times y^2 - y^6 - 121 \times y^4 - 2) + 5.5 \times y^8 + \frac{x}{2 \times y}.$$

Mésaventures du calcul flottant

Soit l'expression

$$333.75 \times y^6 + x^2 \times (11 \times x^2 \times y^2 - y^6 - 121 \times y^4 - 2) + 5.5 \times y^8 + \frac{x}{2 \times y}.$$

L'évaluation de cette expression au point $(x = 77617, y = 33096)$ donne avec la représentation double $1.17260\ldots$, avec la représentation étendue $1.17260\ldots$, alors que la valeur exacte est $-0.82739\ldots$!

Mésaventures du calcul flottant : systèmes nonlinéaires

Soit le système non linéaire provenant de l'économétrie:

$$
\begin{aligned}
&(x_1 + x_1x_2 + x_2x_3 + x_3x_4)x_5 - 1 = 0\\
&(x_2 + x_1x_3 + x_2x_4)x_5 - 2 = 0\\
&(x_3 + x_1x_4)x_5 - 3 = 0\\
&x_4x_5 - 4 = 0\\
&x_1 + x_2 + x_3 + x_4 + 1 = 0
\end{aligned}
$$

La démarche classique :

- Prendre un point initial $x = (1, 1, 1, 1, 1)$ ...
- Puis tenter Newton-Raphson ...

Mésaventures du calcul flottant : systèmes nonlinéaires ... réponses

On pose le problème :

$$
\begin{aligned}
& x_1, x_2, x_3, x_4, x_5 \in [-10^8, +10^8] \\
& (x_1 + x_1x_2 + x_2x_3 + x_3x_4)x_5 - 1 = 0 \\
& (x_2 + x_1x_3 + x_2x_4)x_5 - 2 = 0 \\
& (x_3 + x_1x_4)x_5 - 3 = 0 \\
& x_4x_5 - 4 = 0 \\
& x_1 + x_2 + x_3 + x_4 + 1 = 0
\end{aligned}
$$

On trouve :

solution 1 [safe]
x1 = [9.9701199485551372 7920e-01,1.0031144716396025 4514e+00]
x2 = [9.9434164806817870 7298e-01,1.0061096443788257 7371e+00]
x3 = [9.9075565566375067 8386e-01,1.0101686195471624 1634e+00]
x4 = [-4.0201893393914716 0572e+00,-3.9823207322898905 8578e+00]
x5 = [-1.0040534723085476 8981e+00,-9.9473287065365667 5586e-01]

...

solution 2 [safe]
x1 = [7.9432854389437645981 1e-01,7.9526035060649113983 5e-01]
x2 = [-1.1447110079003592897 4e+00,-1.1436414070305251922 1e+00]
x3 = [3.0245806122477045946 3e-02,3.0784217866707246774 6e-02]
x4 = [-6.8139266434435485919 6e-01,-6.8087604613502150652 2e-01]
x5 = [-5.8748208280603737563 3e+00,-5.8702608831355354013 2e+00]

...

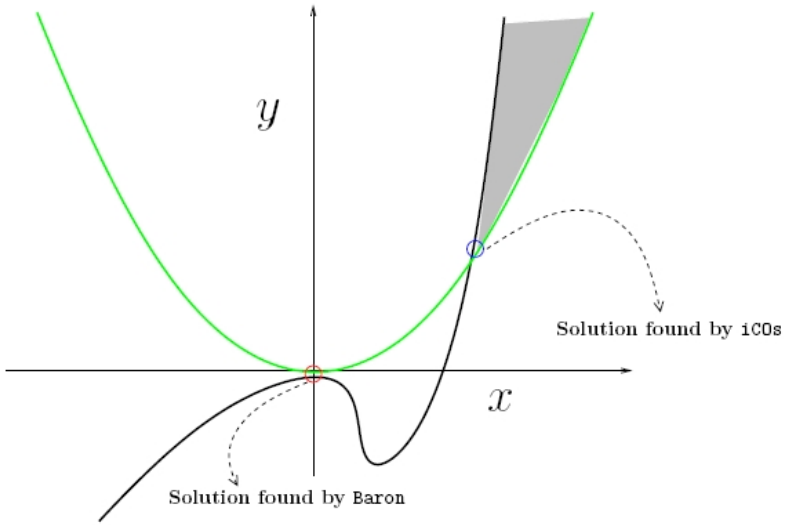
solution 2 [safe]
x1 = [7.9432854389437645 9811e-01,7.9526035060649113 9835e-01]
x2 = [-1.1447110079003592 8974e+00,-1.1436414070305251 9221e+00]
x3 = [3.0245806122477045 9463e-02,3.0784217866707246 7746e-02]
x4 = [-6.8139266434435485 9196e-01,-6.8087604613502150 6522e-01]
x5 = [-5.8748208280603737 5633e+00,-5.8702608831355354 0132e+00]
solution 3 [safe]
x1 = [-4.5314114734273595 8747e-02,-4.4280126689517591 8425e-02]
x2 = [1.6778283472049053 6817e+00,1.6816439297406302 6109e+00]
x3 = [-1.1679650598140476 3475e+00,-1.1657515004918883 5943e+00]
x4 = [-1.4687471107078855 5034e+00,-1.4674757522138646 1486e+00]
x5 = [-2.7257339320998035 2481e+00,-2.7234817292713846 5482e+00]

...

solution 2 [safe]
x1 = [7.9432854389437645 9811e-01,7.9526035060649113 9835e-01]
x2 = [-1.1447110079003592 8974e+00,-1.1436414070305251 9221e+00]
x3 = [3.0245806122477045 9463e-02,3.0784217866707246 7746e-02]
x4 = [-6.8139266434435485 9196e-01,-6.8087604613502150 6522e-01]
x5 = [-5.8748208280603737 5633e+00,-5.8702608831355354 0132e+00]
solution 3 [safe]
x1 = [-4.5314114734273595 8747e-02,-4.4280126689517591 8425e-02]
x2 = [1.6778283472049053 6817e+00,1.6816439297406302 6109e+00]
x3 = [-1.1679650598140476 3475e+00,-1.1657515004918883 5943e+00]
x4 = [-1.4687471107078855 5034e+00,-1.4674757522138646 1486e+00]
x5 = [-2.7257339320998035 2481e+00,-2.7234817292713846 5482e+00]
solution 4 [safe]
x1 = [-2.5000187776073151 6798e-01,-2.4999825683308482 5379e-01]
x2 = [-2.5000204608725673 5163e-01,-2.4999799946553216 6896e-01]
x3 = [-2.5000188729816680 2776e-01,-2.4999806535600160 8617e-01]
x4 = [-2.5000242915396880 8162e-01,-2.4999741910879566 5421e-01]
x5 = [-1.6000138675164233 6037e+01,-1.5999854267077484 5931e+01]

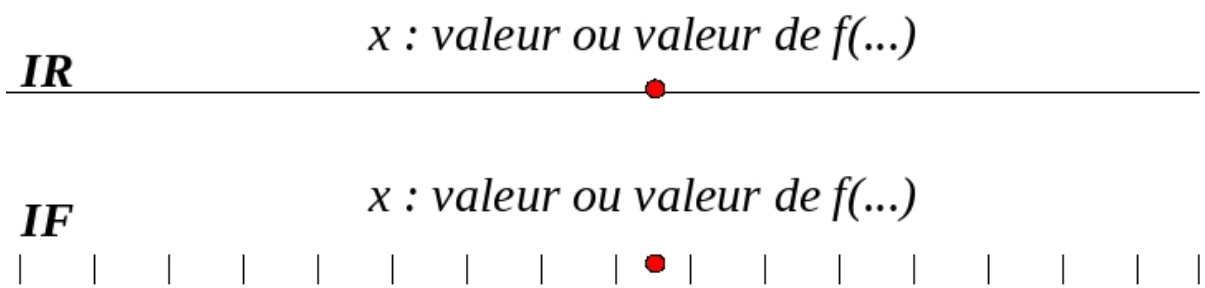
Mésaventures du calcul flottant : un problème d'optim ... réponses ...

$$\begin{array}{ll} \text{minimize} & x \\ \text{subject to} & y - x^2 \geq 0; \quad -y + x^2(x-2) - 10^{-5} \geq 0 \\ & x, y \in [-10, +10] \end{array}$$



Approximation avec la représentation flottante

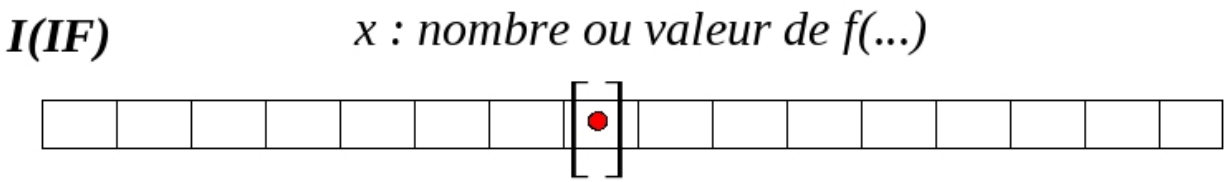
- Etant donné un réel $x \in \boldsymbol{R}$
- x est approximé par un nombre flottant le plus proche
- Propriétés mathématique des réels non respectées



Conséquences :

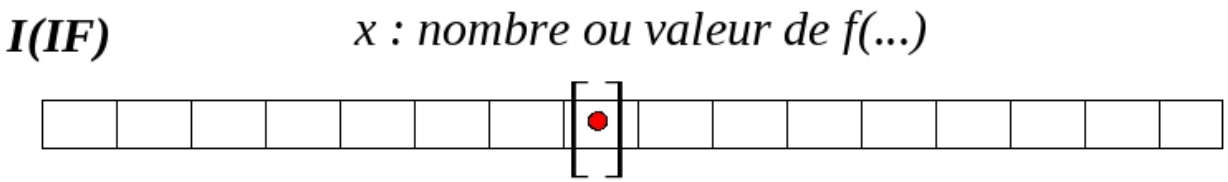
- Les solutions de l'analyse numérique classique sont approchées
- Les solutions peuvent être fausses
- Inexistance de méthodes certifiant la présence d'une solution
- Inéxistance de méthodes pour trouver toutes les solutions

Solution : les intervalles



- Soient α et β deux réelles.
- Encadrer des deux inconnues $\alpha \in A$ et $\beta \in B$ par deux intervalles réels A et B.
- Représentation par les deux plus petits intervalles flottants qui contiennent ces deux intervalles, soit A_f et B_f avec $A \subseteq A_f$ et $B \subseteq B_f$.

Solution : les intervalles



- Soient α et β deux réelles.
- Encadrer des deux inconnues $\alpha \in A$ et $\beta \in B$ par deux intervalles réels A et B.
- Représentation par les deux plus petits intervalles flottants qui contiennent ces deux intervalles, soit A_f et B_f avec $A \subseteq A_f$ et $B \subseteq B_f$.
- Pour toute opération *op* de l'arithmétique réelle nous disposons de son équivalent en arithmétique réelle des intervalles, qui garantit que :

$$(A \ op \ B)_f \subseteq (A_f \ op \ B_f)$$

Solution : analyse par intervalles

En analyse par intervalles, un problème est résolu en trois étapes :

Solution : analyse par intervalles

En analyse par intervalles, un problème est résolu en trois étapes :

1. la théorie est faite sur l'arithmétique des intervalles;

Solution : analyse par intervalles

En analyse par intervalles, un problème est résolu en trois étapes :

1. la théorie est faite sur l'arithmétique des intervalles;
2. le calcul est fait sur l'arithmétique flottante des intervalles;

Solution : analyse par intervalles

En analyse par intervalles, un problème est résolu en trois étapes :

1. la théorie est faite sur l'arithmétique des intervalles;
2. le calcul est fait sur l'arithmétique flottante des intervalles;
3. le principe d'inclusion garantit la validité de la transition de l'arithmétique réelle des intervalles à l'arithmétique flottante des intervalles.

Arithmétique des intervalles, et extension des fonctins

$$[a, b] + [c, d] = [a + c, b + d]$$

Arithmétique des intervalles, et extension des fonctins

$$\begin{aligned} [a,b]+[c,d] &= [a+c, b+d] \\ [a,b]-[c,d] &= [a-d, b-c] \end{aligned}$$

Arithmétique des intervalles, et extension des fonctins

$$
\begin{aligned}
[a,b]+[c,d] &= [a+c, b+d] \\
[a,b]-[c,d] &= [a-d, b-c] \\
[a,b]\times[c,d] &= \\
&\quad [\\
&\quad \min(a\times c, a\times d, b\times c, b\times d), \\
&\quad \max(a\times c, a\times d, b\times c, b\times d) \\
&\quad]
\end{aligned}
$$

Arithmétique des intervalles, et extension des fonctins

$$
\begin{aligned}
[a,b]+[c,d] &= [a+c, b+d] \\
[a,b]-[c,d] &= [a-d, b-c] \\
[a,b]\times[c,d] &= \\
&\quad [\\
&\quad \min(a\times c, a\times d, b\times c, b\times d), \\
&\quad \max(a\times c, a\times d, b\times c, b\times d) \\
&\quad] \\
[a,b]/[c,d] &= [a,b]\times[1/d, 1/c] \ \textbf{if}\ 0\notin[c,d] \\
&\quad [-\infty,+\infty] \ \textbf{if}\ 0\in[c,d]
\end{aligned}
$$

Arithmétique des intervalles, et extension des fonctins

$$
\begin{aligned}
[a,b]+[c,d] &= [a+c, b+d] \\
[a,b]-[c,d] &= [a-d, b-c] \\
[a,b]\times[c,d] &= \\
&\quad [\\
&\quad \min(a\times c, a\times d, b\times c, b\times d), \\
&\quad \max(a\times c, a\times d, b\times c, b\times d) \\
&\quad] \\
[a,b]/[c,d] &= [a,b]\times[1/d, 1/c] \ \textbf{if}\ 0\notin[c,d] \\
&\quad [-\infty,+\infty] \ \textbf{if}\ 0\in[c,d]
\end{aligned}
$$

Il existe des méthodes pour étendre n'importe quelle fonction multivariée sur les intervalles.

Arithmétique des intervalles : exemple

- $\pi = 3.1415926535\ldots$
- $\pi' = [3.1415926534, 3.1415926536]$

- $[1.1, 1.2] + [2.1, 2.2] = [3.3, 3.4]$
- $[10, 10000] + [20, 20000] = [30, 30000]$
- $[1, 2] - [1, 2] = [-1, 1]$?

Arithmétique des intervalles : problème des occurrences multiples

Soit l'expression :

$$e(x) = \frac{x+1}{x}, \text{ avec } x \in \boldsymbol{R}$$

nous pouvons extraire les deux extensions naturelles :

$$e_1(X) = 1 + \frac{1}{X}, \text{ avec } X \in \boldsymbol{I}(\boldsymbol{R})$$
$$e_2(X) = \frac{X+1}{X}, \text{ avec } X \in \boldsymbol{I}(\boldsymbol{R})$$

Arithmétique des intervalles : problème des occurrences multiples

Soit l'expression :

$$e(x) = \frac{x+1}{x}, \text{ avec } x \in \boldsymbol{R}$$

nous pouvons extraire les deux extensions naturelles :

$$e_1(X) = 1 + \frac{1}{X}, \text{ avec } X \in \boldsymbol{I}(\boldsymbol{R})$$
$$e_2(X) = \frac{X+1}{X}, \text{ avec } X \in \boldsymbol{I}(\boldsymbol{R})$$

L'évaluation des deux extensions naturelles sur l'intervalle $[1, 2]$ donne :

$$e_1([1,2]) = 1 + \frac{1}{[1,2]} = [1.5, 2]$$
$$e_2([1,2]) = \frac{[1,2]+1}{[1,2]} = [1, 3]$$

Arithmétique des intervalles : Plus en détail

Transparents "cours-calcul-intervalles.pdf"

Système non linéaires et preuve d'existance

- Extension de la méthode de Newton sur les intervalles : point fixe sur l'opérateur

1: Soit $A_{i,j} = [nat(\frac{\partial f_j}{\partial x_i})(\mathcal{D})], j = 1 \ldots n, i = 1 \ldots n$
2: **for** $i = 1 \ldots n$ **do**
3: $GS_i := (-f_i(mid(\mathcal{D})) - \sum_{j=1, j \neq i}^{n} A_{i,j}(D'_j - mid(D_j)))/A_{i,i} + mid(D_i)$
4: **end for**
5: $\mathcal{D} := \mathcal{D} \cap GS$

Système non linéaires et preuve d'existance

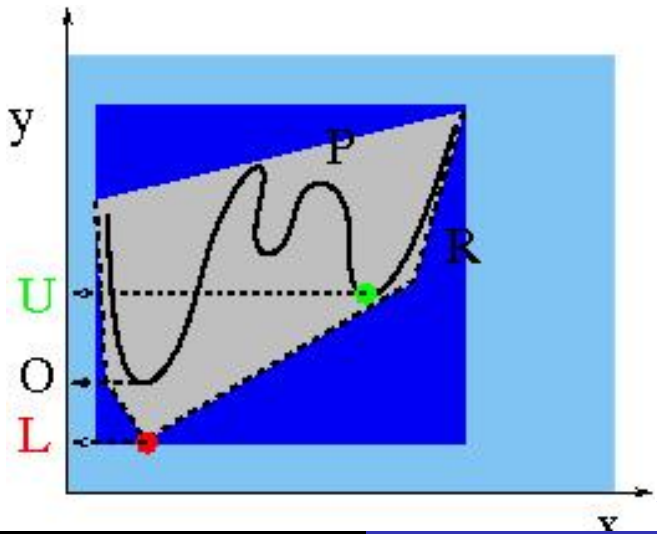
- Extension de la méthode de Newton sur les intervalles : point fixe sur l'opérateur

  1: Soit $A_{i,j} = [nat(\frac{\partial f_j}{\partial x_i})(\mathcal{D})], j = 1 \ldots n, i = 1 \ldots n$
  2: **for** $i = 1 \ldots n$ **do**
  3: $GS_i := (-f_i(mid(\mathcal{D})) - \sum_{j=1, j \neq i}^{n} A_{i,j}(D'_j - mid(D_j)))/A_{i,i} + mid(D_i)$
  4: **end for**
  5: $\mathcal{D} := \mathcal{D} \cap GS$

- Preuve d'existence : S'il existe une itération où après la boucle $GS \subseteq \mathcal{D}$, alors il existe au moins une solution dans la boîte GS.

Algorithme

- Tant que $\mathcal{L} \neq \emptyset$ faire $\%\mathcal{L}$ initialisé avec la boite intiale
  - Sélectionner la boîte B de l'ensemble des boîtes courantes $\mathcal{L}$
  - Réduction (filtrage ou propagation) de B
  - Evaluation inférieure(lower-bounding) de f dans la boîte B
  - Evaluation supérieure (upper-bound) de f dans la boîte B
  - Décomposition de B si elle n'est pas vide; mettre les nouvelles boîtes dans $\mathcal{L}$



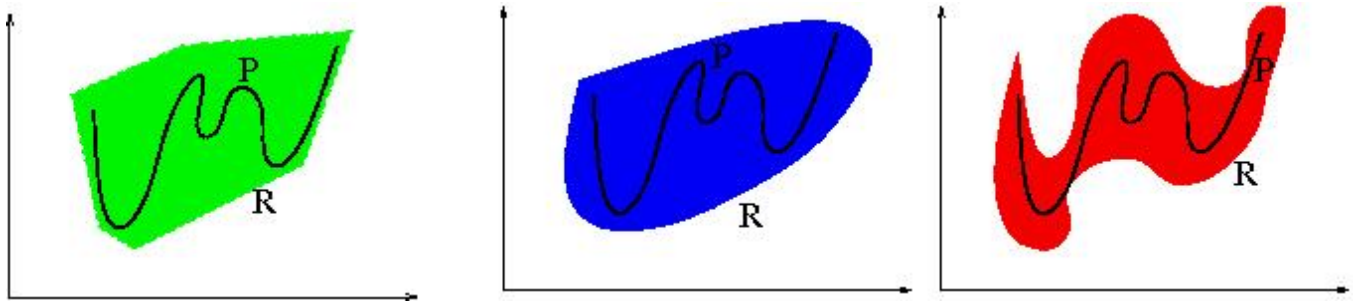
Algorithme

Function BB(IN X, ϵ; OUT $\mathcal{S}$, $[\underline{f}^*, \overline{f}^*]$)
$\mathcal{L} := \{X\}$; $\mathcal{S} := \emptyset$; $(\underline{f}^*, \overline{f}^*) := (-\infty, +\infty)$;
while $w([\underline{f}^*, \overline{f}^*]) > \epsilon$ **do**
  $X' := X''$ tel que $\underline{X}''_{obj} = min\{\underline{X}''_{obj} : X'' \in \mathcal{L}\}$; $\mathcal{L} := \mathcal{L} \backslash X'$;
  $\overline{X}'_{obj} := min(\overline{X}'_{obj}, \overline{f}^*)$;$X' := Reduce(X')$;
  $\underline{X}'_{obj} := LowerBound(X')$;
  $(\overline{X}'_{obj}, X_b, Proved) := UpperBox(X')$;
  if *Proved* **then**
    $\mathcal{S} := \mathcal{S} \cup \{X_b\}$;
  end if
  if $X' \neq \emptyset$ **then**
    $(X'_1, X'_2) := Split(X')$; $\mathcal{L} := \mathcal{L} \cup \{X'_1, X'_2\}$;
  end if
  if $\mathcal{L} = \emptyset$ **then**
    $(\underline{f}^*, \overline{f}^*) := (+\infty, -\infty)$;
  else
    $(\underline{f}^*, \overline{f}^*) := (min\{\underline{X}''_{obj} : X'' \in \mathcal{L}\}, min\{\overline{X}''_{obj} : X'' \in \mathcal{S}\})$;
  end if
end while

Evaluation inférieure et relaxations linéaires

$$\mathcal{R} \equiv \begin{cases} \text{minimize} & f(x) \\ \text{subject to} & g_i(x) = 0, \ i \in \mathcal{RE} \\ & g_i(x) \geq 0, \ i \in \mathcal{RI} \\ & \underline{x_i} \leq x_i \leq \overline{x_i}, x_i \in \mathcal{RX} \end{cases}$$

The objective value at the global minimum of $\mathcal{R}$ is lower than the objective value of the global minimum of $\mathcal{P}$.



Evaluation inférieure et relaxations linéaires

Embed

$$\mathcal{P} \equiv \begin{cases} \text{minimize} & h(x) \\ \text{subject to} & c_i(x) = 0, \ i \in \mathcal{E} \\ & c_i(x) \geq 0, \ i \in \mathcal{I} \\ & \underline{x_i} \leq x_i \leq \overline{x_i}, x_i \in \mathcal{X} \end{cases}$$

into a linear relaxation

$$\mathcal{LR} \equiv \begin{cases} \text{minimize} & cx \\ \text{subject to} & A_E x = b_E, \\ & A_I x \geq b_I, \\ & \underline{x_i} \leq x_i \leq \overline{x_i}, x_i \in \mathcal{RX} \end{cases}$$

Solving $\mathcal{LR}$ provides a rigorous lower bound of the objective function value at the optimum of $\mathcal{P}$, provided that $\mathcal{LR}$ is rigorously generated and solved. The precision of the lower bound depends on the tightness of $\mathcal{LR}$.

Evaluation inférieure et relaxations linéaires

- Construction of the relaxation with RLT (Reformulation Linearisation Techniques) applied to $\mathcal{C} = h(X) \cup c_{i,i \in \mathcal{E} \cup \mathcal{I}}(X)$
  - Let be $\mathcal{I}$ (e.g., x^2, x^n, xy, $x_1 x_2 ... x_n$, $exp(x)$, $log(x)$, ...) the set of nonlinear terms occurring in $\mathcal{C}$.
  - Reformulation: adding new (redundant) constraints $\mathcal{R}$ to take into account the specific semantics of the nonlinear terms I

Evaluation inférieure et relaxations linéaires

- Construction of the relaxation with RLT (Reformulation Linearisation Techniques) applied to $\mathcal{C} = h(X) \cup c_{i,i \in \mathcal{E} \cup \mathcal{I}}(X)$
  - Let be $\mathcal{I}$ (e.g., x^2, x^n, xy, $x_1 x_2 ... x_n$, $exp(x)$, $log(x)$, ...) the set of nonlinear terms occurring in $\mathcal{C}$.
  - Reformulation: adding new (redundant) constraints $\mathcal{R}$ to take into account the specific semantics of the nonlinear terms I
  - Linearisation: generating a set of linear inequalities LR for the whole augmented system $\mathcal{C} \cup \mathcal{R}$ by replacing nonlinear terms by new variables

Evaluation inférieure et relaxations linéaires

- Construction of the relaxation with RLT (Reformulation Linearisation Techniques) applied to $\mathcal{C} = h(X) \cup c_{i,i\in\mathcal{E}\cup\mathcal{I}}(X)$
  - Let be $\mathcal{I}$ (e.g., x^2, x^n, xy, $x_1x_2...x_n$, $exp(x)$, $log(x)$, ...) the set of nonlinear terms occurring in $\mathcal{C}$.
  - Reformulation: adding new (redundant) constraints $\mathcal{R}$ to take into account the specific semantics of the nonlinear terms I
  - Linearisation: generating a set of linear inequalities LR for the whole augmented system $\mathcal{C} \cup \mathcal{R}$ by replacing nonlinear terms by new variables
- Rigorous call to the Simplex algorithm on

$$\mathcal{R} \equiv \begin{cases} \text{minimize} & [f(x)]_I \\ \text{subject to} & LR \\ & \underline{x_i} \leq x_i \leq \overline{x_i}, i = 1..n. \end{cases}$$

Evaluation inférieure et relaxations linéaires

1. Decomposition into a set of quadratic and monovariate terms
2. Relax every quadratic term with classical relaxations
3. Relax every convex term with the Sandwich algorithm

Consider the term $x_1 x_2 x_3 x_4 \exp(x_5)$:

$$\begin{array}{rcl}
\overbrace{x_1 x_2 x_3 x_4 \exp(x_5)}^{y_1} & = & \overbrace{x_1 x_2 x_3}^{y_2} \times \overbrace{x_4 \exp(x_5)}^{y_3} \\
\hline
y_2 & = & \overbrace{x_1 x_2}^{y_4} \times \overbrace{x_3}^{x_3} \\
\hline
y_3 & = & \overbrace{x_4}^{x_4} \times \overbrace{\exp(x_5)}^{y_5} \\
\hline
y_4 & = & \overbrace{x_1}^{x_1} \times \overbrace{x_2}^{x_2} \\
\hline
y_5 & = & \exp(\overbrace{x_5}^{x_5})
\end{array}$$

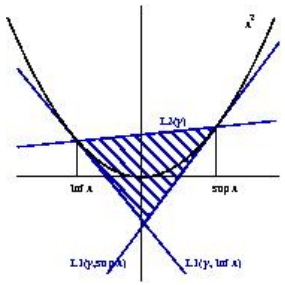
Evaluation inférieure et relaxations linéaires

A power term x^n can be approximated by $n+1$ inequalities [Sherali & al. 1997]:

$$[x^n]_R = \{[(x-\underline{x})^i(\overline{x}-x)^{n-i} \geq 0]_I, i = 0..n\}$$

1. This relaxation generates a tight relations at lower and upper bounds.
2. It generalizes x^2 relaxation on the bounds to x^n.

$$[x^2]_R = \{[(x-\underline{x})^2 \geq 0]_I, [(x-\underline{x})(\overline{x}-x) \geq 0]_I, [(\overline{x}-x)^2 \geq 0]_I\}$$

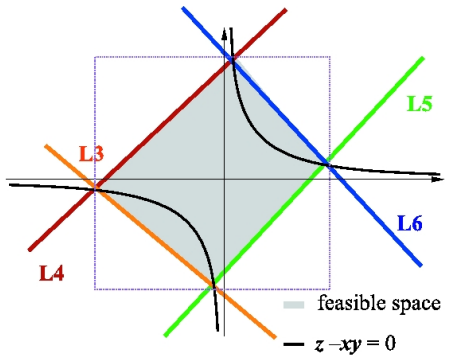


Evaluation inférieure et relaxations linéaires

Soit $g(x_i, x_j) = x_i * x_j$, et $y_{i,j}$ pour $x_i * x_j$

$$L3(y_{i,j}) \equiv [(x_i - \underline{x_i})(x_j - \underline{x_j}) \geq 0]_I; L4(y_{i,j}) \equiv [(x_i - \underline{x_i})(\overline{x_j} - x_j) \geq 0]_I;$$
$$L5(y_{i,j}) \equiv [(\overline{x_i} - x_i)(x_j - \underline{x_j}) \geq 0]_I; L6(y_{i,j}) \equiv [(\overline{x_i} - x_i)(\overline{x_j} - x_j) \geq 0]_I$$

Exemple : $z = x * y, x \in [-5, +5], y \in [-5, +5]$

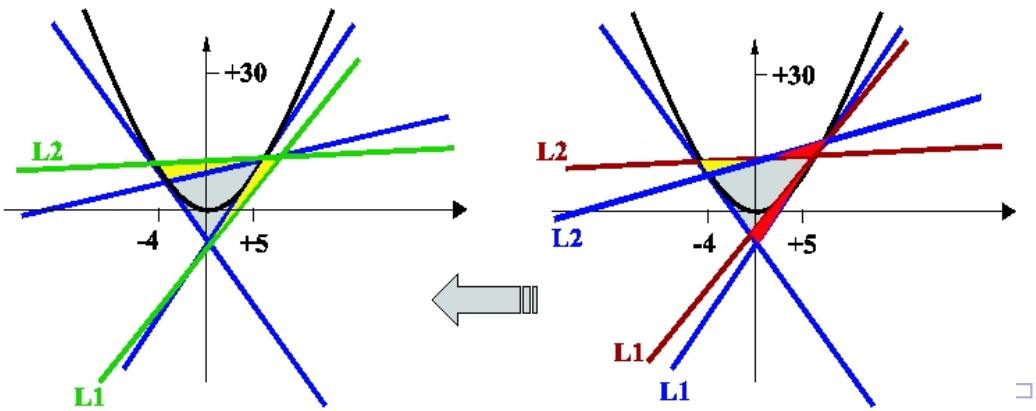


Evaluation inférieure et relaxations linéaires

$$L_1(y,\alpha) \equiv y - 2\alpha x + \alpha^2 \geq 0$$

Idée : augmenter la pente et la constante

$$L_{1F}(y,\alpha) \equiv y - \lfloor 2\alpha \rfloor x + \lceil \alpha^2 \rceil \geq 0 \text{ if } \alpha \geq 0$$
$$L_{1F}(y,\alpha) \equiv y - \lceil 2\alpha \rceil x + \lceil \alpha^2 \rceil \geq 0 \text{ if } \alpha < 0$$



Evaluation inférieure et relaxations linéaires

correction des coefficients

$$\sum_{i=1}^{n} \overline{a_i} x_i + \overline{b} + \overline{\sum_{i=1}^{n} \overline{\overline{a_i}\underline{x_i} - \overline{a_i}\underline{x_i}}} \geq \sum_{i=1}^{n} a_i x_i + b \geq 0, \forall x_i \in x_i.$$

Evaluation inférieure et relaxations linéaires

$$\min \ c^T x \tag{1}$$

$$\text{sous les contraintes} \ \ \underline{b} \leq Ax \leq \overline{b} \tag{2}$$

$$\text{et} \ \ \underline{x} \leq x \leq \overline{x} \tag{3}$$

Le problème : $c^T xr \geq c^T xf$?
Le dual du PL :

$$\max \underline{b}^T z' + \overline{b}^T z'' \tag{4}$$

$$\text{sous les contraintes} \ \ A^T(z' - z'') = c \tag{5}$$

Soit $y = z' - z''$ et le résidu $r = A^T y - c$.
Le résultat fondamental $c^T xr \in (y^T[\underline{b}, \overline{b}] - r^T[\underline{x}, \overline{x}])$
D'où

$$c^T xr > \lfloor y^T[\underline{b}, \overline{b}] - r^T[\underline{x}, \overline{x}] \rfloor \tag{6}$$

Evaluation supèrieure

1. Call a local search method to find an approximate feasible point P_{loc}, possibly nearly optimal (COIN/IpOpt)

$$\mathcal{P} \equiv \begin{cases} \text{minimize} & h(x) \\ \text{subject to} & c_i(x) = 0, \ i \in \mathcal{E} \\ & c_i(x) \geq 0, \ i \in \mathcal{I} \\ & x \in \mathcal{D} \\ & x^0 := P_{lb} \quad \%\text{where } P_{lb} \text{ is the underestimator} \end{cases}$$

2. Prove the feasibility of a box $\mathcal{D}'$ containing P_{loc} on the constraints

$$\begin{aligned} g_i(x) = 0, i \in \{1, \ldots, k\} \\ g_j(x) \leq 0, j \in \{k+1, \ldots, m\} \end{aligned} \tag{7}$$

Evaluation supèrieure

Loop on

1. $\mathcal{D}' = [P_{loc} - \Delta, P_{loc} + \Delta]$
2. Separation of the constraints into two sets: active set and inactive set
   - Inactive set: set of inequalities that are completely satisfied
   - Active set: set of equalities and inequalities that are approximately satisfied (inequalities are considered equalities)
3. We obtain the set of equalities $g_i(x) = 0, i \in \mathcal{AE}$
4. Prove the feasibility of active constraints

Example: Let be $x, y \in [1, 10]$ and the constraint $x + y \geq 0$. With IA, we have $[1, 10] + [1, 10] = [2, 20] \geq 0]$; the constraint is inactive !

Let be the constraint $x - y \geq 0$. With IA, we have $[1, 10] - [1, 10] = [-9, 9]$.

Evaluation supèrieure

1. The Taylor form of the equations is then used to generate the linear system

$$h(c) + \sum_{i=1}^{n} nat(\frac{\partial h}{\partial x_i})(\mathcal{D}')(X_i - c_i) = 0$$

which is equivalent to the $l \times n$ linear interval system

$$A(X - c) = b \tag{8}$$

2. Total pivoting is applied on (8).
3. The last remaining $l - n$ variables are then fixed to their midpoints in order to obtain the squared interval linear system

$$A'(X' - c') = b' \tag{9}$$

4. An existence test is then applied (Krawczyck, Miranda, Borsuk)

Réduction des domaines

Two main classes:

- Feasibility based reduction
  $\Longrightarrow$ discards subdomains where the constraints are not satisfied
  $\Longrightarrow$ optimization, constraint satisfaction (solving equations)
- Optimality based reduction
  $\Longrightarrow$ discards subdomains where the objective function takes values above the current upper bound
  $\Longrightarrow$ optimization

Réduction des domaines

$$\begin{cases} X+Y-1=0 \\ Y-R^*Z=0 \end{cases} \mapsto \begin{cases} (1) & X=1-Y \\ (2) & Y=1-X \\ (3) & Y=R^*Z \\ (4) & Z=\dfrac{Y}{R} \end{cases}$$

- **X∈[-4, 5]; Y ∈[3, 100]; Z ∈[1, 2]; R∈[2, 2]**

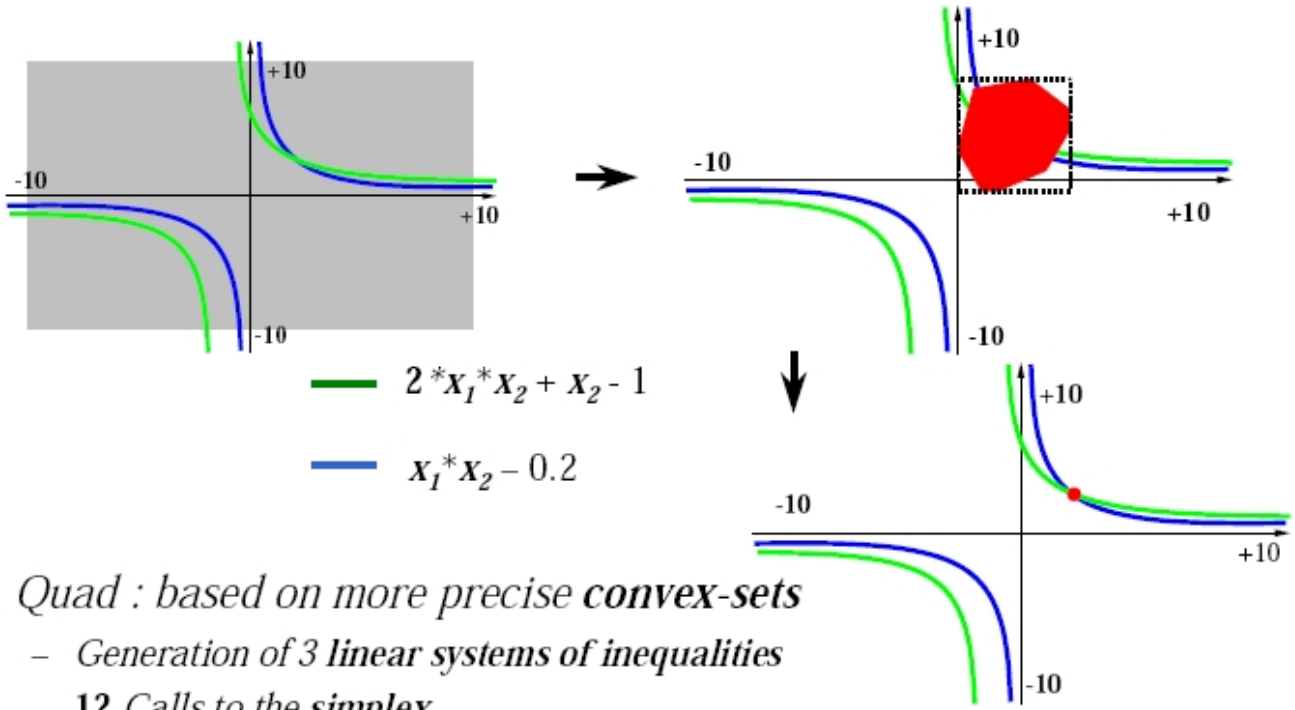
- **(2) → Y=[3, 5]**
- **(4) → Z=[1.5, 2]**
- **(1) → X=[-3,-2]**
- **…**

(1) → X=[-4, -2]
(3) → Y=[3, 4]
…

Réduction des domaines : Plus en détail

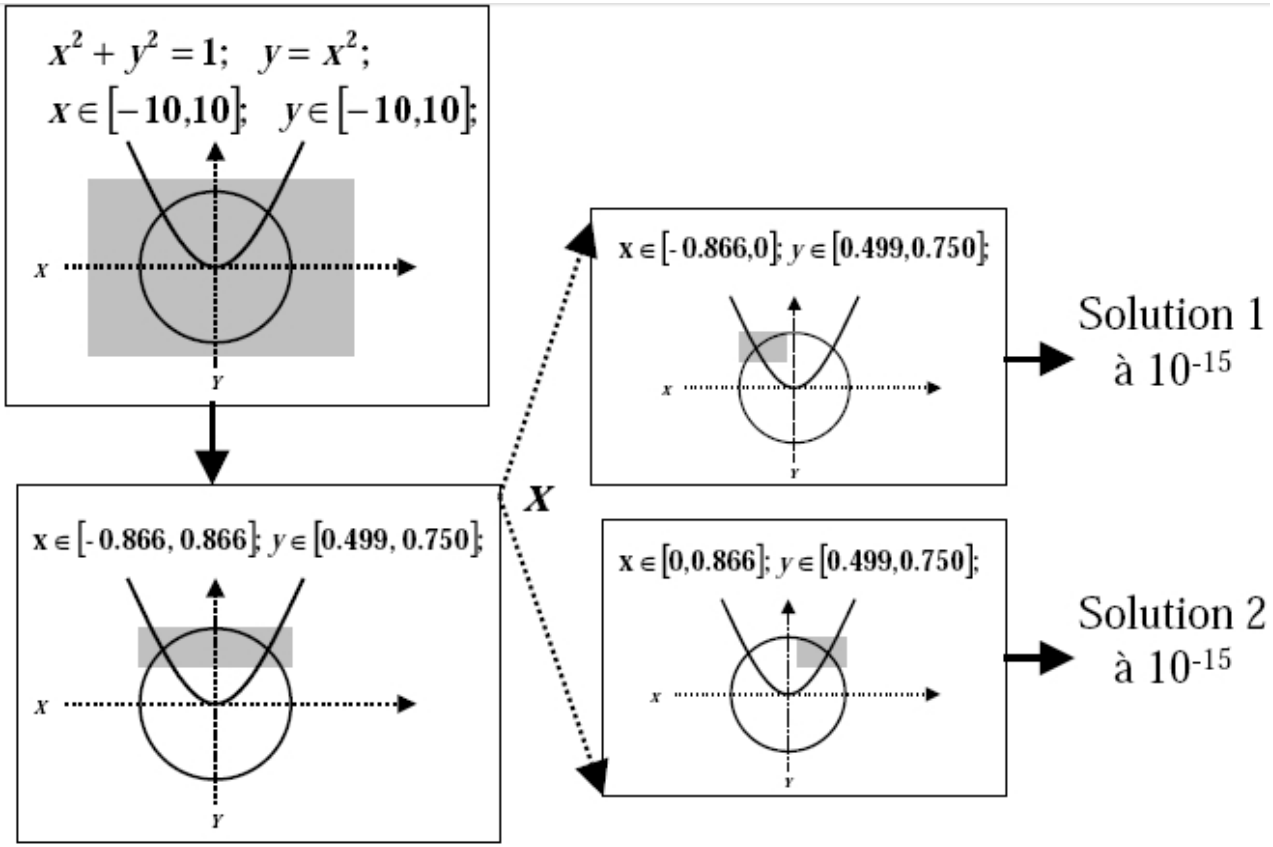
Transparents "cours-consistances-continues-2B.pdf"
Transparents "cours-box-consistance.pdf"

Réduction des domaines



- *Quad : based on more precise* ***convex-sets***
  - *Generation of 3* ***linear systems of inequalities***
  - **12** *Calls to the* ***simplex***
  - *Provides* ***exact solutions***

Illustration d'une résolution d'équations nonlinéaires



Hill climbing

...

Recherche Taboue

...

Récuit simulé

...

Recherches locales avancées

...

Environnement Choco

...

Environnement Gecode

...

Environnement AMPL

ICOS Solver

- General and safe solver: fully rigorous
- Techniques: CP techniques (week and strong propagation), LP interface (Cplex, Coin/Clp), Interval techniques (Interval Newton), Optimization (Branch and bound, linear relaxations), AMPL local solvers interface (ipopt, minos, ...), ...
- AMPL language interface

```
icos example.mod C=strategy.cfg
```

- http://www.essi.fr/~lebbah/icos/

Références

Des figures et des formules de ce cours ont été prises de documents réalisés par :

- Jean-Charles Régin, http://www.constraint-programming.com/people/regin/index.html, contraintes alldiff et gcc
- Arnaud Gotlieb, http://www.irisa.fr/lande/gotlieb/, exemple cas de test logiciel
- Pedro Messeguer, http://www2.iiia.csic.es/ pedro/, figures des 4 reines
- Nicolas Beldiceanu, http://www.emn.fr/x-info/ppc, exemple du problème d'ordonnancement
- Michel Rueher, et Claude Michel, les travaux en commun en optimisation globale